Distributed Semantic Trajectory Similarity Search

Shenghao Gong, Ziquan Fang, Liu Liu, Yaofeng Tu, Yunjun Gao, *Senior Member, IEEE*

***Abstract*—The proliferation of location-based services has facilitated the generation of extensive semantic trajectories for moving objects. In this paper, we investigate the problem of semantic trajectory similarity search, a fundamental task of trajectory analysis. The objective is to identify "similar" trajectories to a query trajectory by considering both spatial proximity and semantic similarity. Existing studies predominantly utilize keywords-based discrete semantic modeling, which fails to capture broader semantics, such as text descriptions. Moreover, these studies typically develop search algorithms on a single machine, resulting in limited processing capability.**

To this end, we propose an effective framework to perform distributed semantic trajectory similarity searches. We introduce a novel method for representing semantic trajectories by employing both geographical sequences and semantic sequences. This approach enhances spatial-semantic awareness in trajectory modeling. Next, we implement the query framework in Spark and propose a carefully designed two-phase, computation-aware partitioning architecture. This is the first architecture to consider both semantic and spatial aspects of trajectories, as well as inter-trajectory distances, to guide the partitioning process. It enables efficient pruning of most partitions in the cluster when processing queries. Within each local partition, we construct an ST-HNSW index to further accelerate queries. Our framework supports three widely used trajectory distance measures: DTW, LCSS, and EDR. Extensive experiments on two real and one synthetic datasets demonstrate the significant advantages of our design, achieving efficiency improvements of up to 1 to 3 orders of magnitude over baseline and alternative approaches.

***Index Terms*—Trajectory Similarity Search, Distributed Computation, Query Optimization**

I. Introduction

With the proliferation of GPS devices, a large number of trajectories of vehicles and individuals have been generated. Trajectory similarity search is a fundamental functionality in trajectory databases [1]–[3], and is useful in transportation and security fields, e.g., finding vehicles that share similar routes to the query abnormal driving vehicle; finding close contacts to the patient's trajectory. Besides, it is a fundamental operator for many downstream analyses including clustering [4], pattern mining [5], and route planning [6]. Overall, the community has devoted efforts into spatial-aware trajectory similarity search [3], [7]–[12], concentrating on searching for similar trajectories considering spatial perspective.

Recently, the explosion of social networks and online maps has augmented trajectories with semantic data [13], [14]. A semantic trajectory is denoted by a sequence of locations where each location is associated with semantic label(s) like keywords and text descriptions. With semantic trajectories, we can know not only where a user has been, but also what he/she has done. In this paper, we investigate semantic trajectory similarity search, which aims to find "similar" trajectories considering spatial-semantic perspective. Overall, semantic trajectory similarity search is extensively used in online maps and social platforms like Google Maps [15] and Foursquare [16]. There are also a multitude of semantic-based travel planning apps, e.g., Wanderlog [17], Rome2Rio [18], etc. Their behind idea is shown in Fig. 1, where three trajectories exist, i.e., τ_1–τ_3. Although τ_1 and τ_2 are spatially close, they express distant semantics and are thus preferred by different groups. In contrast, τ_1 and τ_3 are semantically close and may be recommended to the same group, like city tourists. Consequently, it is important to consider spatial proximity and semantic similarity when determining whether two trajectories are similar to each other. However, when revisiting semantic trajectory analyses [13], [14], [19]–[21], we identify three unsolved challenges in terms of diverse semantic modeling, scalable data processing, and query efficiency, as below.

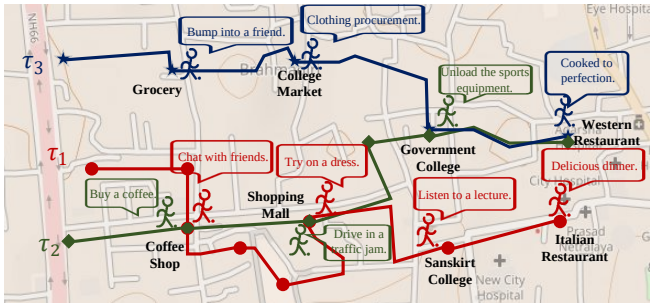


Fig. 1. A Toy Example of Three Semantic Trajectories

Challenge I: How to model diverse semantic meanings? As shown in Fig. 1, semantic trajectories exhibit diverse meanings. For example, τ_1 records someone who chronically "*Chat with friends*" at the Coffee Shop, "*Try on a dress*" at the Shopping Mall, "*Listen to a lecture*" at the Sanskrit College, and had dinner at the Italian Restaurant while commenting "*Delicious dinner*". As observed, two types of semantics exist, i.e., keywords and text descriptions, where the former records users' locations and the latter records users' activities. Note that, even if τ_1 and τ_2 go through the same location, it is possible to express different user activities. For example, τ_1 records someone who "*Chat with friends*" at the Coffee Shop while τ_2 "*Buy a coffee*" without meeting anyone. Besides, with the development of 5G technologies and location based services (LBS), semantic information has become increasingly diverse. However, existing studies [13], [19], [22] mostly focus on keyword-based semantic modeling and cannot effectively model text descriptions. In the above example, existing

S. Gong and Z. Fang are with the School of Software Technology, Zhejiang University, Ningbo 315048, China, E-mail: {gongshenghao, zqfang}@zju.edu.cn.

L. Liu and Y. Gao (Corresponding Author) are with the College of Computer Science, Zhejiang University, Hangzhou 310027, China, E-mail: {liu2, gaoyj}@zju.edu.cn.

Yaofeng Tu is with the ZTE Corporation, Nanjing 210012, China, E-mail: tu.yaofeng@zte.com.cn.

methods directly model the activity "*Chat with friends*" with discrete keywords including "*Chat*", "*with*", and "*friends*". Leaving aside the disruption of the semantic coherence of sentences, we cannot enumerate all keywords. To address this challenge, we propose a new method to model a semantic trajectory into a geospatial sequence and a semantic sequence, respectively. In a semantic sequence, we embed semantic texts into vectors, capturing the rich and deep semantics. Then, we combine the spatial and semantic similarity into a spatial-semantic similarity metric, while supporting popular measures including DTW [23], LCSS [24], and EDR [25].

Challenge II: How to design a scalable distributed semantic trajectory similarity search framework? Compared to raw trajectories that consist of locations, semantic trajectories introduce semantic events, leading to a great increase in data sizes. Taking the widely used trajectory datasets T-drive [26] and GeoLife [27] as examples, enriching them with semantics according to online map APIs [15], [28] leads to 10 $\times$ storage increase. Even worse, owing to the intricacy involved in semantic distance computation, the calculation of similarity between semantic trajectories entails higher complexity. According to our experimental study, after incorporating semantic information into T-drive, the processing time required for a random single query on a single machine exceeds 2,000 seconds. As a result, semantic trajectories may easily exceed the storage capacity and computational capability of a single machine, necessitating a distributed and scalable framework. However, most existing distributed trajectory query processing approaches only support spatial-aware distributed computation. They partition trajectory data using methods such as IDs [29], round-robin [3], or techniques based on separated trajectory points and Minimum Bounding Rectangles (MBRs) [30], [31]. However, these methods are unable to partition data based on the semantic information of trajectories. As a result, most trajectories within the same partition are semantically non-similar, leading to significant redundant computation. On the other hand, existing semantic trajectory query methods do not support large-scale distributed processing and do not have pruning ability (i.e., partition skipping) during partitioning. To address this challenge, we propose a two-phase and computation-aware partitioning architecture over Spark [32]. In contrast to existing trajectory partitioning methods that typically use IDs [29], round-robin [3], or methods based on separated trajectory points and Minimum Bounding Rectangles (MBRs) [30], [33] to perform direct trajectory partitioning, we leverage both the fine-grained semantic and spatial features of trajectories, as well as the inter-trajectory distances to enable two-phase and computation-aware distributed partitioning.

Challenge III: How to accelerate the query process? With the above computation-aware distributed data partitioning, the input trajectory dataset is assigned to different partitions. Given a query trajectory, it will be broadcast into relevant partitions for parallel searching. This brings the idea of designing an effective index in each partition for local pruning. Existing studies typically build inverted indexes over keywords and manage trajectories with multiple inverted files. In each file, they build traditional trajectory indexes, including tree indexes [34], [35] and grid indexes [36], [37]. However, the traditional indexes cannot reduce the time complexity of scanning a partition, but only reduce the number of evaluated trajectories. To address this challenge, we extend the HNSW index [38], which was proposed for vector retrieval, for semantic trajectory similarity search. Specifically, in each data partition, we organize trajectories into a nearest neighbor graph by connecting trajectories that are close in spatial-semantic distance with edges. Based on that, we build a semantic trajectory oriented HNSW index, called ST-HNSW, where the query starts at a node in the graph and iteratively traverses its neighboring nodes. Besides, we also develop a series of optimization techniques on ST-HNSW for further acceleration.

Overall, we revisit semantic trajectory similarity search from semantic modeling, distributed partitioning, and optimized query. And we make the following contributions.

- We propose an expressive semantic trajectory modeling method. Then, we adopt a linear combination way to calculate the spatial-semantic inter-trajectory similarities, while supporting three trajectory distance measures.
- We build a distributed framework for semantic trajectory similarity search, incorporating a novel two-phase, computation-aware partitioning strategy, which enables efficient query processing by allowing the pruning of most irrelevant partitions when a query is executed.
- We develop the ST-HNSW index in each data partition for local query acceleration. Besides, a series of optimization techniques is designed to further speed up local queries.
- Extensive experiments on three large-scale datasets demonstrate the effectiveness, efficiency, and scalability of the proposed framework over competitors.

II. Preliminaries

A. Problem Statement

When reviewing existing semantic trajectory representation methods, there are **individual-based** forms [13], [14], [19], [22], [39] and **global-based** forms [40], [41]. The definitions of the two forms of semantic trajectory are as follows.

Definition 1. (Individual-based Semantic Trajectory) *An individual-based semantic trajectory is an ordered sequence of points, i.e.,* $\tau = \langle p_1, p_2, ..., p_n \rangle$*, where each point* p_i *(*$1 \leq i \leq n$*) contains a location* l_i *and an optional keyword set* $\mathcal{K}_i$*. Here* $\mathcal{K}_i = \{key_1, key_2, ...\}$ *is used to describe the attributes of the corresponding location* p_i*.*

Definition 2. (Global-based Semantic Trajectory) *A global-based semantic trajectory is an ordered point sequence associated with a description keyword set, i.e.,* $\tau = \{\langle p_1, p_2, ..., p_n \rangle, \mathcal{K}_i\}$*, where* $\mathcal{K} = \{key_1, key_2, ...\}$ *is used to describe the attributes of the whole trajectory.*

Under the definition of semantic trajectories mentioned above, STSS aims to find other similar trajectories in a semantic trajectory dataset for a given query trajectory. The definition is as follows.

Definition 3. (Semantic Trajectory Similarity Search, STSS) *Given a set of individual-based or global-based semantic trajectories* $\mathcal{T}$*, a query trajectory* τ_q*, a positive integer*

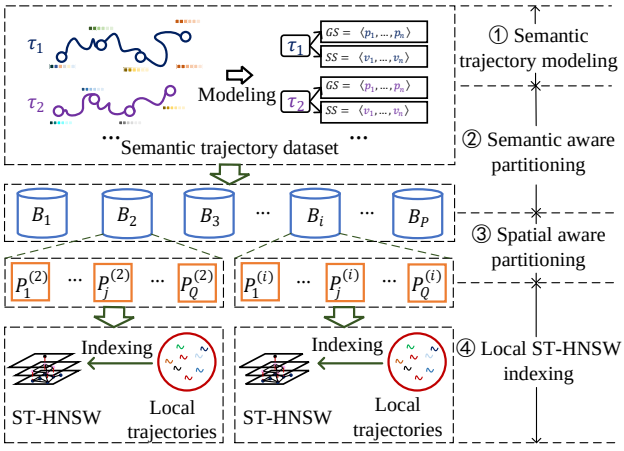


Fig. 2. The Framework Overview

k, and a semantic trajectory distance function $\mathcal{D}(\cdot,\cdot)$, a semantic trajectory similarity search search returns a set of trajectories $\mathcal{R} \subset \mathcal{T}$, where $|\mathcal{R}| = k$ and $\forall \tau_i \in \mathcal{R}, \forall \tau_j \in \mathcal{T} - \mathcal{R}, \mathcal{D}(\tau_j, \tau_q)| = \mathcal{D}(\tau_i, \tau_q)$.

Here $\mathcal{D}(\cdot,\cdot)$ represents commonly used distance measures between semantic trajectories. When higher $\mathcal{D}(\cdot,\cdot)$ indicates higher similarity, "$|=$" signifies "$\leq$". While a higher value of $\mathcal{D}(\cdot,\cdot)$ indicates lower similarity, "$|=$" signifies "$\geq$". Since trajectory representation, indexing, and query methods may involve approximations, many current approaches [11], [42]–[44] produce inherently approximate results, which means they do not strictly satisfy the final inequality condition in Definition 3.

B. Framework Overview

The key to building an efficient distributed framework lies in data partitioning so that on a query trajectory, most of the partitions can be quickly skipped. In this way, idle partitions can be utilized to handle more queries, thereby reducing the query latency caused by query accumulation.

Fig. 2 gives the framework overview, including ①semantic trajectory modeling, ②semantic aware partitioning, ③spatial aware partitioning, and ④local ST-HNSW indexing. The input trajectories are modeled into two sequences. Then, trajectories are partitioned into P **data batches** via semantic features. For each data batch, we further distribute it into Q **data partitions** via spatial features. Based on the above two-phase and computation-aware partitioning, data batches or data partitions that are distant can be excluded. Finally, in each local partition, we build an ST-HNSW index to speed up local queries.

III. Semantic Trajectory Modeling

A. Semantic Trajectory Representation

As observed by Definition 1 and 2, both individual-based and global-based representations rely on **raw keywords**. On the one hand, the semantics are not limited to keywords but also contain semantics such as text descriptions, which cannot be precisely delivered by keywords. Such semantics can be easily obtained in large quantities from reality by utilizing the check- in feature on social platforms and combining it with the API of online maps [15], [28]. On the other hand, as shown in Fig. 3(b), existing studies [13], [14], [22] typically enumerate all keywords to build inverted indexes, resulting in massive redundancy. To this end, we propose a new semantic trajectory representation manner. We first introduce the semantic event.

Definition 4. (Semantic Event, e) *A semantic event e is a tuple, $e = (SR, SA)$. $SR = \{key_1, key_2, ...\}$ is a keyword set describing the attributes of a semantic region. $SA = \{a_1, a_2, ...\}$ is a set of activities that happened at SR.*

Semantic region refers to a set of keywords describing a circular geographical area. These keywords are typically descriptive words that characterize the geographic region, such as "shopping mall" or "sports center." Such keywords are often derived from annotations on online digital maps [15], [28].

Semantic activity denotes a collection of sentences that describe the activities performed by a trajectory within the current semantic region, such as "chat with friends" or "listen to a lecture." These activity descriptions generally originate from social media check-in data, generated through user-published posts and updates.

Definition 5. (Semantic Sequence, SS) *A semantic sequence SS of a trajectory τ is consists of m ordered semantic events, i.e., $\tau.SS = \langle e_1, e_2, ..., e_m \rangle$, where e is defined above.*

Definition 6. (Geographical Sequence, GS) *A geographical sequence GS of a trajectory τ is consists of n points, i.e., $\tau.GS = \langle p_1, p_2, ..., p_n \rangle$, where p is a 2-dimensional location.*

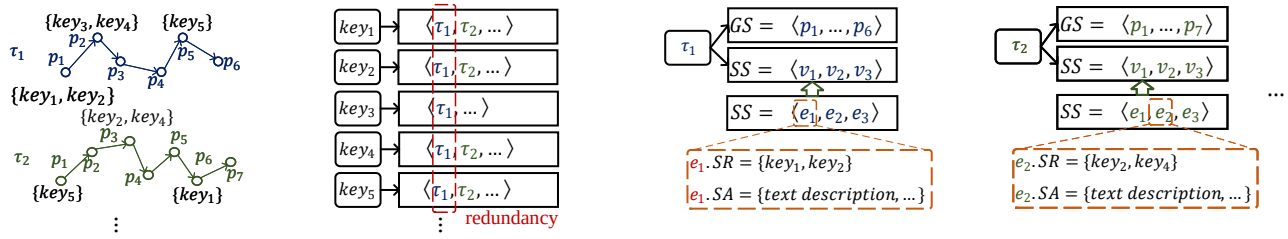
Definition 7. (Semantic Trajectory, τ) *A semantic trajectory is denoted by $\tau = \langle GS, SS \rangle$, where GS denotes τ's geographic sequence and SS denotes τ's semantic sequence.*

Given a raw trajectory, we model it from geographical and semantic aspects. Because the spatial points and semantic events do not strictly a one-to-one correspondence. By decoupling the spatial aspect from the semantic aspect, we can flexibly describe users' mobility features.

To enable robust semantic modeling, we apply Sentence-BERT [45] to embed SRs and SAs into d-dimensional vectors, i.e., $SR \rightarrow \overrightarrow{SR} \in \mathbb{R}^d$, $SA \rightarrow \overrightarrow{SA} \in \mathbb{R}^d$. Then, for each semantic event e_i, we obtain its embedding $\boldsymbol{v_i} = \overrightarrow{e_i.SR} + \overrightarrow{e_i.SA}$. Recall Definition 5, given a semantic sequence $SS = \langle e_1, e_2, ..., e_m \rangle$, we transform it into a vector sequence $SS = \langle \boldsymbol{v_1}, \boldsymbol{v_2}, ..., \boldsymbol{v_m} \rangle$. Fig. 3(c) shows the process.

Semantic event distance. We use $\boldsymbol{v}[i]$ to denote the i-th dimension of $\boldsymbol{v}$ ($1 \leq i \leq d$). For two semantic events e_a and e_b, as well as their vectors v_a and v_b, the semantic event distance is $d(e_a, e_b) = \sqrt{\sum_{i=1}^{d} (v_a[i] - v_b[i])^2}$. By default, we treat semantic sequences as embedded sequences.

Although there are some semantic trajectory similarity embedding [11], [42]–[44] directly embed the whole trajectory. However, we choose to embed semantic text rather than spatial points. First, current embedding methods for semantic trajectories that include semantic text often result in misalignment between the semantic and spatial aspects, which reduces the effectiveness of search, particularly when the semantic text contains complex sentences or exhibits sparse semantic information. Second, the spatial sequence of a trajectory is a relatively simple two-dimensional sequence; embedding it into vectors would introduce unnecessary model training overhead



(a) Individual-based semantic trajectories (b) Inverted-based modeling (c) Our methods

Fig. 3. Semantic Trajectory Representation

in distributed computing scenarios. Finally, embedding spatial sequences would cause trajectories to lose their inherent spatial pruning capability and spatial interpretability.

B. Trajectory Distance Functions

Our focus is not to investigate different similarity measures. Instead, we adopt the well-known DTW [23], LCSS [24], and EDR [25] for spatial-semantic distance.

Definition 8. (Trajectory Distance Function) *Given two trajectories τ_i and τ_j, their inter-distance $\mathcal{D}(\tau_i, \tau_j)$ is defined as the linear combination of $\mathcal{D}^{(G)}(\tau_i, \tau_j)$ and $\mathcal{D}^{(S)}(\tau_i, \tau_j)$:*

$$\mathcal{D}(\tau_i, \tau_j) = \alpha \cdot \mathcal{D}^{(G)}(\tau_i.GS, \tau_j.GS) + (1-\alpha) \cdot \mathcal{D}^{(S)}(\tau_i.SS, \tau_j.SS) \quad (1)$$

Here $\mathcal{D}^{(G)}(\cdot,\cdot)$ denotes the distance between their geo-sequences, $\mathcal{D}^{(S)}(\cdot,\cdot)$ denotes the distance between their semantic sequences, and α balances the weight between them.

The value of α is primarily determined based on the user's personalized requirements. For example, in scenarios where semantic information plays a major role, such as travel route recommendations, α can be appropriately increased to assign greater weight to semantic information in the distance calculation. Conversely, in applications where spatial information is more critical, such as traffic monitoring, α can be decreased to emphasize spatial information. If the user has no specific preference, a balanced weighting between semantic and spatial information should be maintained by setting α to 0.5. It is important to note that both semantic and spatial distances must be normalized before being weighted.

IV. Computation-Aware Partitioning

Given a semantic trajectory dataset $\mathcal{T} = \{\tau_1, \tau_2, ..., \tau_n\}$ where each τ consists of a geo-sequence and a semantic sequence, we first distribute $\mathcal{T}$ into P data batches, i.e., $\mathcal{T} \rightarrow \mathcal{T}' = \{B_1, B_2, ..., B_P\}$. Then we distribute each B_i ($1 \leq i \leq P$) into Q data partitions, i.e., $B_i = \{P_1^{(i)}, P_2^{(i)}, ..., P_Q^{(i)}\}$, where $P_j^{(i)}$ ($1 \leq j \leq Q$) denotes the j-th data partition of the i-th data batch. Overall, $\mathcal{T} \rightarrow \mathcal{T}' \rightarrow \mathcal{T}'' = \{P_1^{(1)}, ..., P_Q^{(1)}; P_1^{(2)}, ..., P_Q^{(2)}; ...; P_1^{(P)}, ..., P_Q^{(P)}\}$.

A. Semantic Aware Partitioning

As shown in Fig. 4, given $\mathcal{T} = \{\tau_1, ..., \tau_n\}$, its semantic event vectors consist of $\mathcal{V} = \{v | v = \overrightarrow{e.SR} + \overrightarrow{e.SA} \ \& \ e \in \tau.SS \ \& \ \tau \in \mathcal{T}\}$. Then, we cluster $\mathcal{V}$ into P groups with DPiSAX [46], i.e., $G_1, ..., G_P$. Based on that, we partition $\mathcal{T}$ into P data batches, i.e., $\mathcal{T} \rightarrow \mathcal{T}' = \{\hat{B}_1, ..., \hat{B}_P\}$, where $\hat{B}_i = \{\tau | \tau \in \mathcal{T} \ \& \ \exists v \in \tau.SS \ \& \ v \in G_i\}$ and $1 \leq i \leq P$. Obviously, this partitioning approach reduces the storage redundancy of the classic inverted-based partitioning methods. In this subsection, we first introduce the DPiSAX-based partitioning method designed for the semantic trajectories modeled above, followed by the distributed search and pruning methods under this partitioning mechanism. Since semantic events are embedded into vectors to capture their rich semantic information (Section 3.1), we selected the DPiSAX algorithm, which currently offers the best partitioning effectiveness in distributed environments without any loss of accuracy while maintaining load balancing. Existing partitioning methods for vectors include Hash-based [47], quantization-based [48], clustering-based [49], tree index-based [50], and graph index-based [38] index approaches. Except for tree-based index methods, all others suffer from significant accuracy loss. Meanwhile, tree-based index methods lack scalability in distributed scenarios and are far less efficient in partitioning compared to DPiSAX.

1) DPiSAX-based partitioning: Specifically, we adopt DPiSAX [46] for semantic event vector clustering, which uses the iSAX encoding to compress vectors into sequences consisting of binary strings. According to the previous work [51], given two semantic vectors $v_a \in \mathbb{R}^d$ and $v_b \in \mathbb{R}^d$, as well as their iSAX embeddings $iSAX(v_a) = \langle x_1, ..., x_\omega \rangle$ and $iSAX(v_b) = \langle y_1, ..., y_\omega \rangle$, the distance between them is

$$d(iSAX(v_a), iSAX(v_b)) = \sqrt{\frac{d}{\omega}} \sqrt{\sum_{i=1}^{\omega} d(x_i, y_i)} \quad (2)$$

Here $d(x_i, y_i)$ denotes the $L2$ distance between two iSAX elements, ω is the length of the *iSAX* sequences, and $\omega \ll d$.

Lemma 1. *Given two event vectors $v_a \in \mathbb{R}^d$ and $v_b \in \mathbb{R}^d$ with their iSAX representations $iSAX(v_a)$ and $iSAX(v_b)$, $d(v_a, v_b) = \sqrt{\sum_{i=1}^{d} (v_a[i] - v_b[i])^2}$ is the vector distance between them. Then we have $d(v_a, v_b) \geq d(iSAX(v_a), iSAX(v_b))$.*

Proof. The proof can be found elsewhere [51]. □

According to Lemma 1, given two semantic events, their $iSAX$ distance is the lower bound of their semantic event distance and thus can be used for pruning unnecessary inter-vector computation. Besides, the computation of $d(iSAX(v_a), iSAX(v_b))$ is far more efficient than the computation of $d(v_a, v_b)$ due to $\omega \ll d$. Based on the vector clustering results, we distribute the dataset into P data batches, i.e., $\mathcal{T} \rightarrow \mathcal{T}' = \{\hat{B}_1, ..., \hat{B}_P\}$.

Consequently, in each data batch $\hat{B}$, all trajectories share the same iSAXs, i.e., $\forall v_a, v_b \in \hat{B}, iSAX(v_a) = iSAX(v_b)$. Thus, we refer to the iSAX code of $\hat{B}$ as $iSAX(\hat{B})$. Then, we define the distance between a trajectory and a data batch.

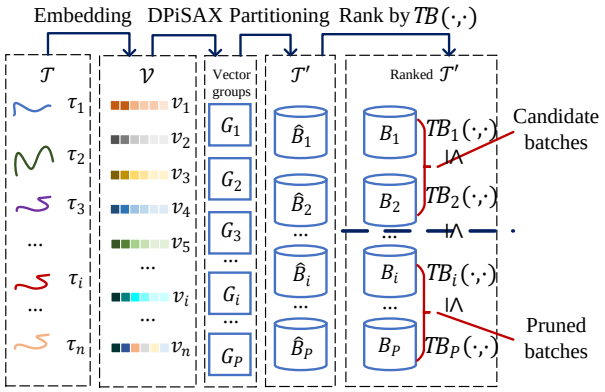


Fig. 4. The Semantic Aware Partitioning

Definition 9. (Trajectory-to-Batch Distance) *Given a trajectory τ and a data batch $\hat{B}$, we define their distance as*

$$TB(\tau, \hat{B}) = \min_{v \in \tau.SS} d(iSAX(v), iSAX(\hat{B})) \tag{3}$$

2) Data batch pruning in search: Based on Definition 9, given a query trajectory τ_q and a data batch $\hat{B}$, we are inspired to derive the relations between $TB(\tau_q, \hat{B})$ and the inter-trajectory semantic distance $\mathcal{D}^{(S)}(\tau_q, \tau_i)$ to prune all trajectories $\tau_i \in \hat{B}$.

Given two semantic sequences $G = \langle g_1, g_2, ..., g_n \rangle$ and $S = \langle s_1, s_2, ..., s_m \rangle$, the LCSS distance of them is calculated as

$$\begin{aligned} LCSS(G,S) &= \sum_{c=1}^{C} \mathcal{W}^c(g_i, s_j), \mathcal{W}^c(g_i, s_j) \\ &= \begin{cases} 1 & \text{if } d(g_i, s_j) \le \varepsilon \ \& \ |i-j| \le \delta \\ 0 & otherwise \end{cases} \end{aligned} \tag{4}$$

Here C denotes the number of matched element pairs from G and S, and $\mathcal{W}^c(\cdot,\cdot)$ denotes the weight of c-th matched pair. If G and S are semantic sequences, $d(g_i, s_j)$ denotes vector-to-vector distance; if G and S are geo-sequences, $d(g_i, s_j)$ denotes point-to-point distance. DTW and EDR can be reformulated similarly.

We unify the weight $\mathcal{W}$ of the distance functions.

$$\mathcal{W}(d(\cdot,\cdot)) = \begin{cases} d(\cdot,\cdot) & \text{if } \mathcal{D} \text{ is } DTW \\ \begin{cases} 1 & \text{if } d(\cdot,\cdot) \le \varepsilon \\ 0 & otherwise \end{cases} & \text{, if } \mathcal{D} \text{ is } LCSS \\ \begin{cases} 1 & \text{if } d(\cdot,\cdot) \ge \varepsilon \\ 0 & otherwise \end{cases} & \text{, if } \mathcal{D} \text{ is } EDR \end{cases} \tag{5}$$

Here $d(\cdot,\cdot)$ denotes the element-pair distance.

Lemma 2. *Given a query trajectory τ_q, a trajectory measure $\mathcal{D}$, and a partitioned trajectory dataset $\mathcal{T}' = \{\hat{B}_1, ..., \hat{B}_P\}$. Then $\forall \hat{B} \in \mathcal{T}'$, $\forall \tau \notin \bigcup_{\hat{B}' \in \mathcal{T}' \& TB(\tau_q, \hat{B}') < TB(\tau_q, \hat{B})} \hat{B}'$, we have $\mathcal{D}(\tau_q.SS, \tau.SS) \ | = \mathcal{W}(TB(\tau_q, \hat{B}))$.*

Proof. Taking $\mathcal{D}$ is DTW as an example. According to Equations 7 and 9, the inter-trajectory distance is determined by the distance between one or several item pairs among the sequences. Thus, $\forall \hat{B} \in \mathcal{T}'$ and $\forall \tau \in \hat{B}$, $\mathcal{D}(\tau_q.SS, \tau.SS) \ge \mathcal{W}(\min_{v_i \in \tau.SS} \min_{v_j \in \tau_q.SS} d(v_i, v_j))$. When $\tau \notin \bigcup \hat{B}'$ where $\hat{B}' \in \mathcal{T}'$ & $TB(\tau_q, \hat{B}') < TB(\tau_q, \hat{B})$, according to Definition 9, we can derive that $\forall v_i \in \tau.SS$, $TB(\tau_q, \hat{B}) \le \min_{v_j \in \tau_q.SS} d(iSAX(v_i), iSAX(v_j))$, that is $TB(\tau_q, \hat{B}) \le \min_{v_i \in \tau.SS} \min_{v_j \in \tau_q.SS} d(iSAX(v_i), iSAX(v_i))$. Based on Lemma 1, $\min_{v_i \in \tau.SS} \min_{v_j \in \tau_q.SS} d(iSAX(v_i), iSAX(v_i)) \le \min_{v_i \in \tau.SS} \min_{v_j \in \tau_q.SS} d(v_i, v_j)$. Overall, we have $\mathcal{W}(TB(\tau_q, \hat{B})) \le \mathcal{D}(\tau_q.SS, \tau.SS)$. EDR and LCSS measures share a similar proof process and thus are omitted. □

Based on Lemma 2, we show how to prune irrelevant data batches. Given a query trajectory τ_q, a distance function $\mathcal{D}$, and a partitioned dataset $\mathcal{T}'$, we rank $\mathcal{T}'$ such that $TB(\tau_q, B_i) \le TB(\tau_q, B_j)$ when $i \le j$. Here, $1 \le i \le P, 1 \le j \le P$, $B_i \in \mathcal{T}'$, and $B_j \in \mathcal{T}'$. Fig. 4 shows the ranking process. Assuming the current search batch is B_i, if $\mathcal{W}(TB(\tau_q, B_i))$ satisfies the query criteria, e.g., $\mathcal{W}(TB(\tau_q, B_i))$ has exceeded the distance between τ_q and the farthest trajectory in returned trajectories on the scanned data batches B_j $(1 \le j \le i)$, the data batches ranked behind B_i can be pruned.

B. Spatial Aware Partitioning

1) Envelope:

Definition 10. (MBR Sequence) *Given a geo-sequence $GS = \langle p_1, p_2, ..., p_n \rangle$ and an order constraint $\delta \in \mathbb{Z}$, we transform GS into an MBR sequence $MS = \langle MBR(p_1), MBR(p_2), ..., MBR(p_n) \rangle$, and $MBR(p_i) = (x_i^l, y_i^l, x_i^u, y_i^u)$ where*

$$\begin{aligned} x_i^l &= \min\{x_j | i-\delta \le j \le i+\delta\}, \\ y_i^l &= \min\{y_j | i-\delta \le j \le i+\delta\}, \\ x_i^u &= \max\{x_j | i-\delta \le j \le i+\delta\}, \\ y_i^u &= \max\{y_j | i-\delta \le j \le i+\delta\} \end{aligned} \tag{6}$$

As shown in Fig. 5(a), when $\delta = 2$, the blue dashed box is the MBR of p_2, the green dashed box is the MBR of p_3, and the purple dashed box is the MBR of p_4.

Definition 11. (MBR-to-MBR Distance) *Given two MBRs $M_i = MBR(x_i^l, y_i^l, x_i^u, y_i^u)$ and $M_j = MBR(x_j^l, y_j^l, x_j^u, y_j^u)$, their distance is $d(M_i, M_j) = \sqrt{d_x^2 + d_y^2}$, where*

$$d_x = \begin{cases} 0 & \text{if } [x_i^l, x_i^u] \cap [x_j^l, x_j^u] \ne \varnothing \\ \min(x_j^u - x_i^l, x_i^u - x_j^l) & \text{otherwise} \end{cases} \tag{7}$$

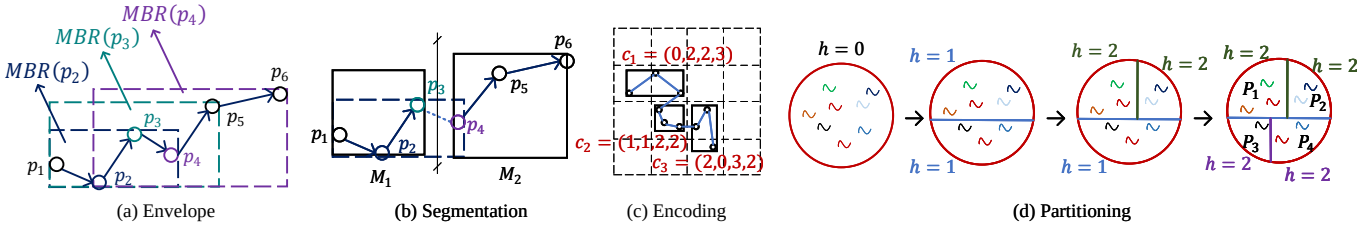
$$d_y = \begin{cases} 0 & \text{if } [y_i^l, y_i^u] \cap [y_j^l, y_j^u] \ne \varnothing \\ \min(y_j^u - y_i^l, y_i^u - y_j^l) & \text{otherwise} \end{cases} \tag{8}$$

Definition 12. (MBR-sequence Distance) *Given two geo-sequences GS_a and GS_b as well as their MBR sequences $MS_a = \langle MBR(p_1^a), ..., MBR(p_m^a) \rangle$ and $MS_b = \langle MBR(p_1^b), ..., MBR(p_n^b) \rangle$, assume that $m \le n$, we define their distance as*

$$\begin{aligned} MSD(MS_a, MS_b) = &\sum_{i=1}^{m} \mathcal{W}(d(MBR(p_i^a), MBR(p_i^b))) + \\ &\sum_{i=m+1}^{\min(n, m+\delta)} \mathcal{W}(d(MBR(p_m^a), MBR(p_i^b))) \end{aligned} \tag{9}$$

Lemma 3. *Given GS_a, GS_b, and their MBR sequences MS_a, MS_b, then $\mathcal{D}(GS_a, GS_b) \ | = MSD(MS_a, MS_b)$.*

Proof. Taking $\mathcal{D}$ is DTW as an example. Given $p_i \in GS_a$ & $p_j \in GS_b$, we get $d(p_i, p_j) \ge d(MBR(p_i), MBR(p_j))$. Assume that the length of GS_a and GS_b are m and n, where $m \le n$, $\mathcal{D}(GS_a, GS_b) = \sum_{c=1}^{C} \mathcal{W}^c \ge \sum_{i=1}^{m} \mathcal{W}\ (d(MBR(p_i^a),$



(a) Envelope (b) Segmentation (c) Encoding (d) Partitioning

Fig. 5. The Spatial Aware Partitioning

$MBR(p_i^b))) + \sum_{i=m+1}^{\min(n,m+\delta)} \mathcal{W}\ (d(MBR(p_m^a), MBR(\ p_i^b)))$. Then $\mathcal{D}(GS_a, GS_b) \geq MSD(\ MS_a, MS_b)$. The above proof still holds when $n \leq m$. EDR and LCSS measures share a similar proof and thus are omitted. □

Based on Lemma 3, given two trajectories, we first envelope them into MBR sequences. Then, we compute the distance of them, which can be used to prune unnecessary distance computation. To accelerate the distance computation, given an MBR sequence *MS*, we segment *MS* as below.

2) Segmentation:

Definition 13. (MBR Sequence Segmentation) *Given a geo-sequence* $GS = \langle p_1, p_2, ..., p_n \rangle$, *its MBR sequence* $MS = \langle MBR(p_1), MBR(p_2), ..., MBR(p_n) \rangle$, *and a segment length* $\mathcal{L}$, *we segment MS into a sequence* MS' *that consists of* $\frac{n}{\mathcal{L}}$ *MBRs, i.e.,* $MS' = \langle M_1, M_2, ..., M_{\frac{n}{\mathcal{L}}} \rangle$, *where* $M_i = (sx_i^l, sy_i^l, sx_i^u, sy_i^u)$ $(1 \leq i \leq \frac{n}{\mathcal{L}})$ *is computed as*

$$sx_i^l = \min_{j=\mathcal{L}(i-1)+1}^{\mathcal{L}i} x_j^l, \quad sy_i^l = \min_{j=\mathcal{L}(i-1)+1}^{\mathcal{L}i} y_j^l$$
$$sx_i^u = \max_{j=\mathcal{L}(i-1)+1}^{\mathcal{L}i} x_j^u, \quad sy_i^u = \max_{j=\mathcal{L}(i-1)+1}^{\mathcal{L}i} y_j^u \tag{10}$$

Overall, given a geo-sequence *GS* of p points, we first envelope *GS* into an MBR sequence *MS* of p MBRs, i.e., $GS = \{p_1, p_2, ..., p_n\} \rightarrow MS = \{MBR(p_1), MBR(p_2), ..., MBR(p_n)\}$. Then, we segment *MS* into a simplified MBR sequence that consists of $\frac{n}{\mathcal{L}}$ MBRs, i.e., $MS \rightarrow MS' = \{M_1, M_2, ..., M_{\frac{n}{\mathcal{L}}}\}$, where the length of each MBR is $\mathcal{L}$. As shown in Fig. 5(b), 2 MBRs are enough to bound the geo-sequence.

3) Encoding:

Definition 14. (MBR Sequence Encoding) *Given a geo-sequence GS of n points, its segmented MBR-sequence* $MS' = \langle M_1, ..., M_i, ..., M_{\frac{n}{\mathcal{L}}} \rangle$, *and a geo-span* (x_l, x_r, y_d, y_u), *we encode* MS' *into* $\mathtt{MS} = \langle c_1, ..., c_i, ..., c_{\frac{n}{\mathcal{L}}} \rangle$, *where* $c_i = (c_i^l, c_i^d, c_i^r, c_i^u) = (\lfloor \frac{sx_i^l - x_l}{(x_r - x_l)/2^\omega} \rfloor, \lfloor \frac{sy_i^l - y_d}{(y_u - y_d)/2^\omega} \rfloor, \lceil \frac{sx_i^u - x_l}{(x_r - x_l)/2^\omega} \rceil, \lceil \frac{sy_i^u - y_d}{(y_u - y_d)/2^\omega} \rceil)$, *and* ω *controls the length of the binary code.*

Specifically, for each MBR M_i in MS', we compute its code $c_i = (c_i^l, c_i^d, c_i^r, c_i^u)$, where c_i^l and c_i^d denote the bottom left location of M_i while c_i^r and c_i^u denote top right location. In Fig. 5(c), when $\omega = 2$, we split the space into 4×4 grids. The trajectory is segmented to an MBR sequence that consists of 3 MBRs, where the codes of MBRs are $c_1 = (0, 2, 2, 3)$, $c_2 = (1, 1, 2, 2)$, and $c_3 = (2, 0, 3, 2)$.

Definition 15. (Encoded Sequence Distance) *Given two encoded MBR-sequences* $\mathtt{MS}_a = \langle c_1^a, ..., c_{sa}^a \rangle$ *and* $\mathtt{MS}_b = \langle c_1^b, ..., c_{sb}^b \rangle$, *assume* $sa \leq sb$, *the distance between them is*

$$EMSD(\mathtt{MS}_a, \mathtt{MS}_b) = \sum_{i=1}^{sa} \mathcal{W}(d(c_i^a, c_i^b)) + \sum_{i=sa}^{\min(sa+\lceil \delta/\mathcal{L} \rceil, sb)} \mathcal{W}(d(c_{sa}^a, c_i^b)) \tag{11}$$

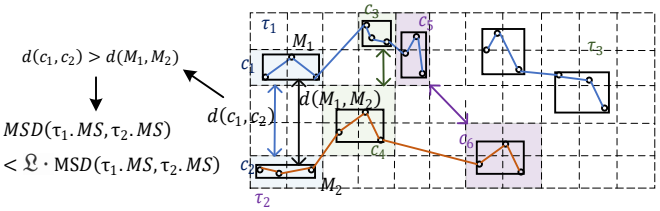


Fig. 6. A Toy Example for Lemma 4

Lemma 4. *Given two MBR sequences* MS_a, MS_b *of two geo-sequences, and their encoded MBR sequences* $\mathtt{MS}_a$, $\mathtt{MS}_b$. *When* $\mathtt{MS}_a$ *and* $\mathtt{MS}_b$ *have the same fixed length* $\mathcal{L}$ *of MBR segment, then we have* $MSD(MS_a, MS_b) \mid = \mathcal{L} \cdot EMSD(\mathtt{MS}_a, \mathtt{MS}_b)$.

Proof. Taking $\mathcal{D}$ is DTW as an example. Given a geo-sequence GS_a with length m, and its encoded MBR sequence $\mathtt{MS}_a$ with length *sa*, according to Equation 10, $\forall i \in [1, sa]$ and $\forall j \in [\mathcal{L}(i-1)+1, \mathcal{L}i]$, we have $[x_j^l, x_j^u] \subseteq [sx_i^l, sx_i^u]$ and $[y_j^l, y_j^u] \subseteq [sy_i^l, sy_i^u]$. Then we derive that $MBR(p_j^a) \subseteq M_i = (sx_i^l,\ sy_i^l,\ sx_i^u,\ sy_i^u)$. Similarly, according to definition 14, we have $M_i \subseteq c_i^a$. Therefore $MBR(p_j^a) \subseteq c_i^a$. For another GS_b with length b, and its encoded MBR sequence $\mathtt{MS}_b$ with length *sb*, we also derive that $MBR(p_j^b) \subseteq c_i^b$. Assume that $m < n$, then $\forall i \in [1, sa]$ and $\forall j \in [\mathcal{L}(i-1)+1, \mathcal{L}i]$, $d(MBR(p_j^a), MBR(p_j^b)) \geq d(c_i^a, c_i^b)$, i.e., $\sum_{i=\mathcal{L}(i-1)+1}^{\mathcal{L}i} d(MBR(\ p_j^a), MBR(\ p_j^b)) \geq \mathcal{L} \cdot d(c_i^a,\ c_i^b)$. Therefore, we have $\sum_{i=1}^{sa} \sum_{j=\mathcal{L}(i-1)+1}^{\mathcal{L}i} d(MBR(p_j^a), MBR(p_j^b)) \geq \sum_{i=1}^{sa} \mathcal{L} \cdot d(c_i^a, c_i^b)$, i.e., $\sum_{i=1}^{m} d(MBR(\ p_i^a),\ MBR(\ p_i^b)) \geq \mathcal{L} \sum_{i=1}^{sa} d(c_i^a,\ c_i^b)$. Similarly, we derive that $\sum_{i=m+1}^{\min(m+\delta,n)} d(MBR(\ p_m^a),\ MBR(\ p_i^b)) \geq \mathcal{L} \sum_{i=sa}^{\min(sa+\lceil \delta/\mathcal{L} \rceil, sb)} d(c_m^a, c_i^b)$. Then $MSD(GS_a, GS_a) = \sum_{i=1}^{m} \mathcal{W}\ (\ d(MBR(p_i^a), MBR(p_i^b))) + \sum_{i=m+1}^{\min(m+\delta,n)} \mathcal{W}\ (\ d(MBR(\ p_m^a), MBR(\ p_i^b))) \geq \mathcal{L} \cdot \sum_{i=1}^{sa} \mathcal{W}\ (\ d(c_i^a, c_i^b)) + \sum_{i=sa}^{\min(sa+\lceil \delta/\mathcal{L} \rceil, sb)} \mathcal{W}(\ d(\ c_{sa}^a,\ c_i^b))$. Therefore, we have $MSD(\ GS_a,\ GS_b) \geq \mathcal{L} \cdot EMSD(\mathtt{MS}_a, \mathtt{MS}_b)$. EDR and LCSS share a similar proof process and are omitted. □

Example. As shown in Fig. 6, during the distance calculation between trajectories τ_1 and τ_2, according to Definition 12, $MSD(\tau_1.MS, \tau_2.MS)$ is the sum of the distances between all MBRs, while $EMSD(\tau_1.\mathtt{MS}, \tau_2.\mathtt{MS})$ is the sum of the distances between the minimum grid ranges of the MBRs. Since an MBR must lie within its corresponding grid range, $EMSD(\tau_1.\mathtt{MS}, \tau_2.\mathtt{MS})$ serves as a lower bound for $MSD(\tau_1.MS, \tau_2.MS)$. Moreover, since the minimum grid range of each MBR contains $\mathcal{L}$ MBRs, Lemma 4 holds.

4) Partitioning:

Definition 16. (Partition Code of an MBR) *Given an encoded and segmented MBR sequence* $\mathtt{MS}$, *for each MBR* $c_i = (c_i^l, c_i^d, c_i^r, c_i^u) \in \mathtt{MS}$, *the partition code of* c_i *is defined as* $pc_i = (pc_i^l, pc_i^d, pc_i^r, pc_i^u)$, *where* $pc_i^l = c_i^l \gg (\omega - 1)$, $pc_i^d = c_i^d \gg (\omega - 1)$, $pc_i^r = c_i^r \gg (\omega - 1)$, $pc_i^u = c_i^u \gg (\omega - 1)$.

Given a geo-sequence *GS*, its MBR sequence $\texttt{MS} = \langle c_1, ..., c_i, ..., c_{\frac{n}{\mathcal{L}}} \rangle$, we encode its h ($h \leq \frac{n}{\mathcal{L}}$) MBRs using Definition 4.12. Then, we get the partition code for *GS*, i.e., $PC(GS) = \langle pc_1, ..., pc_i, ..., pc_h \rangle$. Therefore, we partition each batch B into Q partitions, i.e., $B = \{P_1, P_2, ..., P_Q\}$, such that for each P_i ($1 \leq i \leq Q$), all its trajectories share the same code, i.e., $\forall \tau_a, \tau_b \in P_i \ \& \ \tau_a \neq \tau_b$, $PC(\tau_a.GS) = PC(\tau_b.GS)$. Thus, we refer to the code of P_i as $PC(P_i)$. In Fig. 5(d), we repeatedly partition the largest until the specified partition number Q is achieved. After each partitioning, h is increased by 1, so h is referred to the partition depth.

5) Data partition pruning in search:

Definition 17. (Trajectory-to-Partition Distance) *Given a data partition P with its code $PC(P) = \langle pc_1, ..., pc_d \rangle$, a geo-sequence GS of n points, and its encoded MBR sequence $\texttt{MS} = \langle c_1, ..., c_{\frac{n}{\mathcal{L}}} \rangle$, where $pc_i = (pc_i^l, pc_i^d, pc_i^r, pc_i^u)$. Assuming the max length of the segmented MBR sequences is smax and the minimum length is smin, when $d < \min(\frac{n}{\mathcal{L}}, smin)$, we define the distance between geo-sequence GS and partition P as*

$$TP(GS, P) = \sum_{i=1}^{d} \mathcal{W}(d(c_i, pc_i)) + (smax - d)\mathcal{W}(0), where$$
$$d(c_i, pc_i) = d(c_i, MBR(2^{\omega-1}pc_i^l, 2^{\omega-1}pc_i^d, 2^{\omega}pc_i^r, 2^{\omega}pc_i^u)) \quad (12)$$

Based on Definition 17, we give a lemma below for data partition pruning during searches.

Lemma 5. *Given a query trajectory τ_q, a trajectory distance function $\mathcal{D}$, and a set of the spatially partitioned data partitions on a data batch (i.e., $B = \{P_1, ..., P_Q\}$). Assume the length of the MBR segment of $\tau_q.GS$ is $\mathcal{L}$. Then $\forall P \in B, \forall \tau \in P$, we have $\mathcal{D}(\tau_q.GS, \tau.GS) \mid = \mathcal{L} \cdot TP(\tau_q.GS, P)$.*

Proof. Recall the partition code of a spatial partition P is $PC(P) = \langle pc_1, ..., pc_d \rangle$, where $pc_i = (pc_i^l, pc_i^d, pc_i^r, pc_i^u)$. $\forall \tau \in P$, $PC(\tau.GS) = PC(P)$. Let $\texttt{MS}_q$ and $\texttt{MS}$ denote the encoded MBR-sequences. According to Definitions 14 and 16, $\forall i \in [1, d]$, assume $c_i = \texttt{MS}.c_i = MBR(c_i^l, c_i^d, c_i^r, c_i^u)$ and $c_i^q = \texttt{MS}_q.c_i$, we derive $[\ c_i^l, c_i^r\ ] \subseteq [\ 2^{\omega-1}pc_i^l, 2^{\omega}pc_i^r\ ]$ and $[\ c_i^d, c_i^u\ ] \subseteq [\ 2^{\omega-1}\ pc_i^d, 2^{\omega}pc_i^u\ ]$. Then $c_i \subseteq MBR(2^{\omega-1}pc_i^l, 2^{\omega-1}pc_i^d,\ 2^{\omega}pc_i^r,\ 2^{\omega}pc_i^u)$. Therefore, $d(c_i^q,\ MBR(2^{\omega-1}pc_i^l, 2^{\omega-1}pc_i^d,\ 2^{\omega}pc_i^r,\ 2^{\omega}pc_i^u)) \leq d(c_i^q,\ c_i)$. According to Definition 17, we have $d(c_i^q,\ MBR(2^{\omega-1}pc_i^l,\ 2^{\omega-1}pc_i^d,\ 2^{\omega}pc_i^r, 2^{\omega}pc_i^u)) = d(c_i^q, pc_i)$. Then $d(c_i^q, c_i) \geq d(c_i^q, pc_i)$. Assume the lengths of $\tau_q.\texttt{MS}$ and $\tau.\texttt{MS}$ are sq and s. When $\min(sq, s) = sq$, we have $\sum_{i=1}^{sq} \mathcal{W}(\ d(c_i^q,\ c_i)) + \sum_{i=sq+1}^{\min(sq+\lceil \delta/\mathcal{L} \rceil, s)} \mathcal{W}((c_i^q, c_i)) \mid = \sum_{i=1}^{d} \mathcal{W}(\ d(c_i^q,\ bc_i)) + (smax\text{-}d)\mathcal{W}(0)$, which means $EMSD(\texttt{MS}_q, \texttt{MS}) \mid = TP(\tau_q.GS, P)$. When $\min(sq, s) = s$, the same result are obtained. According to Lemmas 3 and 4, we have $\mathcal{D}(\tau_q.GS, \tau.GS) \mid = MSD(\tau_q.GS, \tau.GS) \mid = \mathcal{L}\ EMSD(\texttt{MS}_q, \texttt{MS}) \mid = \mathcal{L}\ TP(\tau_q.GS, P)$, that is $\mathcal{D}(\tau_q.GS, \tau.GS) \mid = \mathcal{L}TP(\tau_q.GS, P)$. □

V. Local Index and Query Process

A. Local ST-HNSW Index

1) ST-HNSW Construction: The construction of a semantic-trajectory-oriented HNSW index (ST-HNSW) follows HNSW [38]. In ST-HNSW, each trajectory is treated as a node and inserted into the hierarchical neighbor graph.

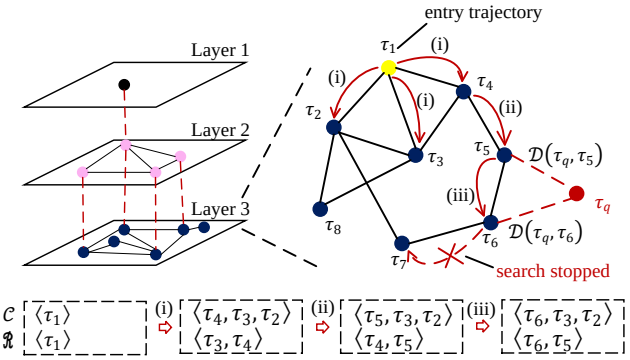


Fig. 7. The Search Process in ST-HNSW

When inserting a trajectory, there is a certain probability of entering a specific layer in the graph. During the insertion, the graph finds the M trajectories nearest to the trajectory being inserted, and the connections are established between them. Overall, the building and searching in ST-HNSW involve k nearest neighbor (kNN) search in a specific layer. As shown in Fig. 7, each layer in ST-HNSW represents a progressively larger NSW (Navigable Small World) [52], with the bottom layer being a complete NSW that contains all trajectories.

2) ST-HNSW Search: We detail how to perform search in a single layer. Fig. 7 shows an example of $k = 2$. Specifically, two priority queues are maintained: $\mathcal{C}$ that stores trajectories in ascending order of the distance to τ_q and $\mathcal{R}$ that stores query results in descending order of distance. We perform kNN as follows: (i) Starting from the entry trajectory τ_1, all neighbor trajectories τ_2, τ_3, and τ_4 of the current trajectory are traversed and enqueued into $\mathcal{C}$. The closer trajectories τ_3 and τ_4 are added to $\mathcal{R}$; (ii) The neighbor trajectory τ_5 of τ_4 is traversed and enqueued into $\mathcal{C}$ and the trajectory τ_5 is added to $\mathcal{R}$; (iii) The neighbor trajectory τ_6 of τ_5 is traversed and enqueued into $\mathcal{C}$ and the trajectory τ_6 is added to $\mathcal{R}$; (iv) The distance from the trajectory at the front of $\mathcal{C}$ to the target trajectory is not less than the distance from the trajectory at the front of $\mathcal{R}$ to the target τ_q, the query stops, and $\{\tau_5, \tau_6\}$ is returned.

B. Query Process

1) Two-Phase based Search: Given a query τ_q and a dataset $\mathcal{T}$, based on the semantic aware partitioning, we obtain P sorted data batches, i.e., $\mathcal{T} \rightarrow \mathcal{T}' = \{B_1, ..., B_P\}$ such that $TB(\tau_q, B_i) \leq TB(\tau_q, B_j)$. Then, based on the spatial aware partitioning, we distribute each data batch $B_i \in \mathcal{T}'$ ($1 \leq i \leq P$) into Q data partitions, i.e., $B_i = \{P_1^{(i)}, ..., P_Q^{(i)}\}$.

Definition 18. (Bound Distance) *Given a query trajectory τ_q, a ranked batches $\mathcal{T}' = \{B_1, ..., B_P\}$ where $B_i = \{P_1^{(1)}, ..., P_Q^{(P)}\}$ ($1 \leq i \leq P$), and the segment length $\mathcal{L}$. The bound distance of τ_q and $P_j^{(i)}$ ($1 \leq j \leq Q, P_j^{(i)} \in B_i$) is*

$$BD(\tau_q, P_j^{(i)}) = \alpha \cdot \mathcal{W}(TB(\tau_q.SS, B_{i-1})) + (1 - \alpha) \cdot \mathcal{L} \cdot TP(\tau_q.GS, P_j^{(i)}) \quad (13)$$

Lemma 6. *Given a query trajectory τ_q, a distance function $\mathcal{D}$, a ranked semantic batches $\mathcal{T}' = \{B_1, ..., B_P\}$ where each data batch $B_i = \{P_1^{(i)}, ..., P_Q^{(i)}\}$ contains Q data partitions, we then have $\forall \tau \in P_k^{(i)} (1 \leq k \leq Q) \& \tau \notin \bigcup_{j<i} B_j$, $\mathcal{D}(\tau_q, \tau) \geq BD(\tau_q, P_k^{(i)})$.*

Proof. The proof is straightforward and follows directly from Lemmas 2 and 5. □

Based on Lemma 6, we can use *BD* to prune unnecessary data batches and data partitions. During similarity search, if $BD(\tau_q, P_k^{(i)})$ has exceeded the distance between τ_q and the farthest trajectory in returned trajectories on searched partitions, the remaining data partitions in B_i are pruned.

2) Optimized ST-HNSW based Search: ST-HNSW computes the distance between two trajectories in both construction and search phases, which cost cannot be ignored. Therefore, we extend HNSW into weighted HNSW and deploy triangle inequality for optimization. When inserting trajectories into ST-HNSW, the distances $\mathcal{D}(\cdot,\cdot)$ between trajectories are saved as the edge weight.

Recall Equation 1, if $\mathcal{D}^{(G)}(\cdot,\cdot)$ and $\mathcal{D}^{(S)}(\cdot,\cdot)$ satisfy triangle inequality, $\mathcal{D}(\cdot,\cdot)$ satisfies triangle inequality. Thus, during the search in each layer of ST-HNSW, we combine weighted edges and triangle inequality to prune.

Lemma 7. *During the process of finding the top k nearest neighbors of the query trajectory τ_q, assuming the current search node is τ_c (i.e., the first trajectory in the candidate queue $\mathcal{C}$ for traversing), τ_n is a neighbor of τ_c and $\mathcal{R}$ is the set of current top k nearest neighbors of τ_q. When $|\mathcal{D}(\tau_c,\tau_n) - \mathcal{D}(\tau_c,\tau_q)| \mid = \max_{\tau\in\mathcal{R}} \mathcal{D}(\tau,\tau_q)$, then $\tau_n \notin \mathcal{R}$.*

Proof. According to $\mathcal{D}(\tau_n,\tau_q) \mid = |\mathcal{D}(\tau_c,\tau_n) - \mathcal{D}(\tau_c,\tau_q)|$, we have $\mathcal{D}(\tau_n,\tau_q) \mid = \max_{\tau\in\mathcal{R}} \mathcal{D}(\tau,\tau_q)$. Then $\tau_n \notin \mathcal{R}$. □

Lemma 8. *During the process of finding the top k nearest neighbors of the query trajectory τ_q, assuming the current search node is τ_c (i.e., the first trajectory in the candidate queue $\mathcal{C}$), τ_n is a neighbor of τ_c and $\mathcal{R}$ is the set of current top k nearest neighbors of τ_q. When $BD(\tau_c,\tau_n) \mid = \mathcal{D}(\tau_c,\tau_q)$ and $BD(\tau_c,\tau_n) - \mathcal{D}(\tau_c,\tau_q) \mid = \max_{\tau\in\mathcal{R}} \mathcal{D}(\tau,\tau_q)$, then $\tau_n \notin \mathcal{R}$.*

Proof. According Lemma 6, $\mathcal{D}(\tau_c,\ \tau_n) \mid = BD(\tau_c,\tau_n)$. Assume that $\mathcal{D}(\tau_c,\tau_n) = BD(\tau_c,\tau_n) + \Delta\mathcal{D}$, $\Delta\mathcal{D} \geq 0$, according to triangle inequality, we have $\mathcal{D}(\ \tau_c,\ \tau_q) + \mathcal{D}(\ \tau_n,\ \tau_q) \mid = \mathcal{D}(\ \tau_c,\ \tau_n)$. Then $\mathcal{D}(\ \tau_n,\ \tau_q) \mid = \mathcal{D}(\ \tau_c,\ \tau_n) - \mathcal{D}(\ \tau_c,\ \tau_q)$, i.e., $\mathcal{D}(\ \tau_n,\ \tau_q) \mid = BD(\ \tau_c,\ \tau_n) + \Delta\mathcal{D} - \mathcal{D}(\tau_c,\tau_q)$. Therefore $\mathcal{D}(\ \tau_n,\ \tau_q) \mid = BD(\tau_c,\tau_n) - \mathcal{D}(\tau_c,\tau_q)$. Then $\mathcal{D}(\tau_n,\tau_q) \mid = \max_{\tau\in\mathcal{R}} \mathcal{D}(\ \tau,\tau_q)$. So we have $\tau_n \notin \mathcal{R}$. □

In ST-HNSW, a fixed entry may increase search steps because the distance between the entry point and its neighbors is far. We use a grid index to find a suitable entry point in advance. Specifically, we divide the space into grids, where each grid contains all trajectories within that region. During searches, we use the grid index to find the adjacent grid regions to the query trajectory, and then select a suitable entry point within that region. This method reduces the number of search steps and improves ST-HNSW efficiency.

Finally, we give the distributed semantic trajectory similarity search in Algorithm 1. Lines 2–4 initialize a result queue for each query, where each queue maintains the current nearest distance values. Then, lines 5–7 calculate the bound distance *BD* between a query trajectory and a data partition. If the bound distance does not violate Lemma 6, we dispatch this query into this partition and perform optimized ST-HNSW search (lines 8–13). If the *TB* distance between the query trajectory to the current data batch violates Lemma 2, we prune all data partitions from this data batch (lines 14–16).

Algorithm 1: Trajectory Similarity Search

Input: a set of queries Q, a distance function $\mathcal{D}$, the parameter k, and a trajectory dataset $\mathcal{T}^{''} = Map(Int, List(List()))$ that stores data batches and data partitions

```
1  R_τ(Int, List()) ← Null // a map to store the results
2  for each query τ_q ∈ Q do
3      R_τ[τ_q.ID] ← PriorityQueue(k)
4      bsfDist ← ∞
5      for each batch B ∈ T'' do
6          for each partition P ∈ B do
7              if bsfDist | = BD(τ_q, P) then
8                  // Definition 18 and Lemma 6
9                  LocalRes ← Optimized ST-HNSW Search //
                     Section V-B
10                 for τ ∈ LocalRes do
11                     R_τ[τ_q.ID].add(τ)
12                 bsfDist ← D(τ_q, R_τ[τ].tail())
13         if α · TB(τ_q, B) | = bsfDist then
14             // Lemma 2
15             break
```

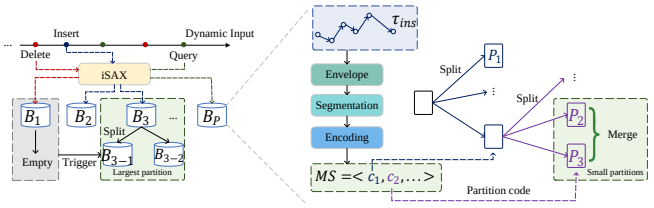
Example. A priority queue is first initialized to store the search results. The query trajectory is then dispatched to batches that satisfy the condition in Lemma 2, and further forwarded to partitions meeting the criterion in Lemma 6. Finally, the ST-HNSW search (Section 5.1) is performed locally, and the results are inserted into the priority queue.

Complexity Analysis. Due to the parallel execution of each partition in a distributed environment, the time complexity of Algorithm 2 is $O(\log \frac{|\mathcal{T}|}{PQ})$ for the ST-HNSW search complexity, where $|\mathcal{T}|$ is the size of the dataset and $\frac{|\mathcal{T}|}{PQ}$ is the average size in each partition. The space complexity includes the space required to store the ST-HNSW index and the space occupied by the trajectory dataset, which is $O(|\mathcal{T}|M + |\mathcal{T}|(|GS| + |SS|))$, where M is the degree of nodes in ST-HNSW, $|SS|$ is the average length of the semantic sequences, and $|GS|$ is the average length of the spatial sequences.

VI. Dynamic Partitioning and Indexing

Based on the existing semantic aware partitioning and spatial aware partitioning, we have designed two extensions: dynamic semantic aware partitioning and dynamic spatial aware partitioning. Specifically, for dynamic semantic aware partitioning, we assign newly added trajectories to batches with the same iSAX codes [51]. based on their semantic sequences, while removing trajectories that need to be deleted from their respective batches. For dynamic spatial aware partitioning, we perform the splitting and merging of partitions according to the current number of trajectories in each partition.

Dynamic semantic aware partitioning. During semantic aware partitioning, trajectories are divided based on the iSAX encoding of their semantic sequences, meaning that trajectories within each batch share the same iSAX code. Therefore, as shown in Fig. 8(a), assuming the current set of semantic batches is $\mathcal{T}^{'} = \{\hat{B_1}, ..., \hat{B_P}\}$, and the iSAX code of each batch is $\{iSAX(\hat{B_1}), ..., iSAX(\hat{B_P})\}$, for



(a) Dynamic Semantic Aware Partitioning (b) Dynamic Spatial Aware Partitioning

Fig. 8. Dynamic Partitioning and Indexing

a newly inserted trajectory τ_{ins}, it should first be encoded into $iSAX(\tau_{ins}.SS) = \{iSAX(v_1), ..., iSAX(v_n)\}$. If $\exists iSAX(v) \in iSAX(\tau_{ins}.SS)$ & $iSAX(v) = iSAX(\hat{B}_i)$, where $B_i \in \mathcal{T}^{'}$, it is inserted into the corresponding batch B_i. For a trajectory τ_{del} that needs to be deleted, its iSAX code $iSAX(\tau_{del}.SS)$ is similarly used to locate the corresponding batch B_j, from which it is then removed. If all trajectories in a batch are deleted, the batch currently containing the largest number of trajectories is split into two sub-batches.

Dynamic spatial aware partitioning. After performing dynamic semantic aware partitioning, the processing of newly added or to-be-deleted trajectories can be further simplified by following the three steps outlined in Section IV-B: envelope, segmentation, and encoding. These trajectories can then be further partitioned using the dynamic spatial partitioning method described below. Similarly, assuming the current set of spatial partitions is $\mathcal{T}^{''} = \{P_1, ..., P_Q\}$, and the partition code (definition 16) of each partition is $\{PC(P_1), ..., PC(P_Q)\}$. As shown in Fig. 8(b) for a newly inserted trajectory τ_{ins}, after envelope, segmentation, and encoding, it should first be encoded into $\texttt{MS} = \langle c_1, c_2, ...\rangle$. If $\texttt{MS} = PC(P_i)$, where $P_i \in \mathcal{T}^{''}$, it is inserted into the corresponding batch P_i. For a trajectory τ_{del} that needs to be deleted, its MBR sequence encoding MS is similarly used to locate the corresponding partition P_j, from which it is then deleted.

The dynamic balance of each partition is maintained according to the steps outlined in Algorithm 2. Specifically, in line 1, the partition depth is initialized for the current spatial partitions. Lines 3–14 repeatedly partition the largest partition in the spatial partition list until the specified partition number Q is achieved (as illustrated in Fig. 8). Lines 4–5 find the biggest partition. Line 6 increases the partition depth. Line 7 deletes the current partition. Line 8–14 create new partitions according to the partition code (Definition 16). Finally, lines 15–17 repeatedly merge the small partitions if there exist two partitions, where the sum of trajectories is less than the average. The time complexity of Algorithm 2 is $O(|GS| \log |\mathcal{T}|)$, where $|\mathcal{T}|$ is the size of the dataset and $|GS|$ is the average length of the spatial sequences.

Then, we introduce a dynamic partitioning strategy based on partition load balancing. Specifically, our goal is to optimize load balance across all partitions. During the partitioning process, we adopt a greedy approach to incrementally achieve this objective. Any partition splitting or merging operation is executed only if it leads to a more balanced load distribution among the partitions.

First, we define the following load balancing metric based on the number of trajectories in each partition.

Algorithm 2: Dynamic Trajectory Partitioning

Input: the set $\mathcal{T}^{''}$ of current spatial partitions, the number Q of spatial partitions

1 $\mathcal{T}^{''}.depth \leftarrow 0$
2 **while** $\mathcal{T}^{''}.partition_num() < Q$ ***or*** $\exists P_1, P_2 \in \mathcal{T}^{''}, |P_1| + |P_2| < |\mathcal{T}^{''}|/Q$ **do**
3 **if** $\mathcal{T}^{''}.partition_num() < Q$ **then**
4 // Split the biggest partition
5 $BP \leftarrow$ the biggest partition in $\mathcal{T}^{''}$
6 $\mathcal{T}^{''}.depth \leftarrow \mathcal{T}^{''}.depth + 1$
7 $\mathcal{T}^{''}$.delete(BP)
8 **for** *each partition code* $c \in [0, 2^{\omega-1})$ **do**
9 $NP \leftarrow \emptyset$ // create a new partition
10 **for** *each trajectory* $\tau \in \mathcal{T}^{''}$ **do**
11 **if** $\tau.\texttt{MS}[0, B.depth] = c$ **then**
12 //Definition 16
13 NP.add(τ)
14 $\mathcal{T}^{''}$.add(NP)
15 **if** $\exists P_1, P_2 \in \mathcal{T}^{''}, |P_1| + |P_2| < |\mathcal{T}^{''}|/Q$ **then**
16 // Merge the partitions smaller than the threshold
  Merge the partition P_1 and P_2
17 $\mathcal{T}^{''}.depth \leftarrow \mathcal{T}^{''}.depth - 1$

Definition 19. (Load Balancing Metric) *For a given set of partitions $\mathcal{T}^{''} = \{P_1, ..., P_i, ...P_Q\}$, its load balancing metric $\sigma(\mathcal{T}^{''})$ is defined as the variance of the trajectory number across all partitions, i.e.,*

$$\sigma(\mathcal{T}^{''}) = \frac{1}{Q}\sum_{i=1}^{Q}(|P_i| - \overline{|P|}) \tag{14}$$

Here Q is the partition number and $\overline{|P|}$ is the average trajectories number of all partitions.

Therefore, when we aim to optimize the load distribution across all partitions, the objective is to minimize the aforementioned load balancing metric, i.e.,

$$(\mathcal{T}^{''})^* = \arg\min_{\mathcal{T}^{''}} \sigma(\mathcal{T}^{''}) \tag{15}$$

In dynamic scenarios, the future distribution of $\mathcal{T}^{''}$ is unknown. Therefore, we can only adopt a greedy strategy to ensure that every partition splitting or merging operation reduces the value of $\sigma(\mathcal{T}^{''})$. Accordingly, an additional condition needs to be incorporated into lines 3 and 15 of Algorithm 2: namely, that the value of $\sigma(\mathcal{T}^{''})$ after merging/deletion must be smaller than its value before the merging/deletion.

VII. Extensions

In this section, we show that our framework supports another two important trajectory similarity searhes, i.e., threshold-based search and time-ordered search.

Algorithm 3: Threshold-based Search

Input: a set of queries Q, a distance function $\mathcal{D}$, the distance threshold θ, and a trajectory dataset $\mathcal{T}^{''} = Map(Int, List(List()))$ that stores data batches and data partitions

1 $R_\tau(Int, List()) \leftarrow Null$ // a map to store the results
2 **for** *each query* $\tau_q \in Q$ **do**
3 $R_\tau[\tau_q.ID] \leftarrow$ List()
4 **for** *each batch* $B \in \mathcal{T}^{''}$ **do**
5 **for** *each partition* $P \in B$ **do**
6 **if** $\theta \mid = BD(\tau_q, P)$ **then**
7 // Definition 18 and Lemma 6
8 LocalRes $\leftarrow$ Threshold-based ST-HNSW Search
9 **for** $\tau \in$ *LocalRes* **do**
10 $R_\tau[\tau_q.ID]$.add(τ)
11 **if** $\alpha \cdot TB(\tau_q, B) \mid = \theta$ **then**
12 // Lemma 2
13 break

A. Threshold-based Search

Unlike kNN search, threshold-based search returns all trajectories in the semantic trajectory dataset that satisfy a given threshold θ. We define it as below.

Definition 20. (Threshold-based Search) *Given a set of trajectories $\mathcal{T}$, a query trajectory τ_q, a distance function $\mathcal{D}$, and a threshold θ, threshold-based similarity search returns a set of trajectories $\mathcal{R} \subset \mathcal{T}$, where $\forall \tau \in \mathcal{R}, \theta \mid = \mathcal{D}(\tau, \tau_q)$.*

Specifically, by replacing the input parameters k and ϵ in Algorithm 2 with θ, and substituting $(1+\epsilon) \cdot$ *bsfDist* with θ in lines 7 and 13, our method can be seamlessly extended to support the above threshould-based search problem. In addition, in the Local Search process, the search result $\mathcal{R}$ is transformed into an unbounded set, and the insertion condition is adjusted to satisfy the threshold θ. The pseudo-code for threshould-based search is shown in Algorithm 3. Lines 2–3 initialize a result list for each query. Then, lines 4–7 calculate the bound distance *BD* between a query trajectory and a data partition. If the bound distance does not violate Lemma 6, we dispatch this query into this partition and perform a ST-HNSW search (lines 8–10). Simultaneously, if the *TB* distance between the query trajectory to the current data batch violates Lemma 2, we directly prune all data partitions from this data batch (lines 11–13).

Complexity Analysis. The time and space complexity of the algorithm is the same as Algorithm 1.

B. Time-ordered Search

Time-ordered search involves conducting similarity searches on time-ordered semantic trajectories, where each coordinate point in a trajectory is accompanied by a timestamp. We redefine Definition 6 as follows.

Definition 21. (Time-ordered Geographical Sequence, *TGS*) *A time-ordered geographical sequence TGS of a trajectory τ is consists of n time-ordered points, i.e., $\tau.GS = \langle p_1, p_2, ..., p_n \rangle$, where p is a 3-dimensional (x, y, t) location including longitude x, latitude y, and timestamp t.*

In the subsequent distance calculations between coordinate points, we replace the original two-dimensional Euclidean distance calculation with a three-dimensional Euclidean distance calculation that incorporates the time dimension.

TABLE I
STATISTICS OF THE DATASETS USED

Attributes	T-drive	Brinkhoff	GeoLife
#trajectories	3,347,280	34,780,800	1,886,640
#GPS points	152,358,760	1,769,973,400	497,539,560
#keywords	133,404,220	1,552,058,600	436,085,220
#descriptions	77,256,139	707,989,360	257,178,733
Disk usage	179.2 GB	3.57 TB	115.2 GB

VIII. EXPERIMENTS

We use research questions below to guide experiments:
RQ1. How does the semantic modeling perform?
RQ2. How does the distributed framework perform?
RQ3. How useful of the proposed pruning technology?
RQ4. How robust is the framework for parameters?
RQ5. Can this framework support dynamic queries?
RQ6. How this framework support other queries?

A. Setup

Hardware. All the experiments were conducted on a cluster consisting of 6 nodes. Each node is equipped with two 12-core processors (Intel Xeon E5-2620 v3 2.40 GHz), 64GB RAM, Ubuntu 14.04.3, and Spark 3.1.1.

Datasets. We use two real datasets, T-Drive [26] and GeoLife [27], as well as a synthetic dataset, Brinkhoff [53]. Specifically, i) T-drive contains taxi trajectories of Beijing, during a period of 7 days; ii) GeoLife contains people trajectories that involve various transportation modes, during a period of more than three years; and iii) Brinkhoff is generated on real road networks of San Joaquin. Following previous works [13], [14], [19], [22], we augment raw trajectory datasets with semantics. By using the coordinates of the check-in data from Weibo and a collection of real-life tweets [54], we associated them with the corresponding coordinates of the above datasets. Table I summarizes the statistics of the datasets used.

Parameters. In the experiments, α balances the importance of spatial versus semantic similarity; $\mathcal{L}$ denotes the segment length of segmented MBR sequences; δ_g denotes the grid size in the grid optimization for ST-HNSW; P and Q are the numbers of batches and partitions; P/Q balances the partition numbers of semantic and spatial partitions; k is used in the search; O_r denotes the percentage of moving objects w.r.t. all objects, and N denotes the number of data partitions after two-layer partitioning.

Baselines. We evaluate our methods from two dimensions, i.e., **semantic modeling** and **search processing**, and compared with following baselines.

A) Semantic modeling comparison. We compare the proposed semantic trajectory representation manner termed `ST` (Eq. 1, supported by LCSS, DTW, and EDR) with existing measures in terms of effectiveness and robustness. Specifically, we compare `ST` with i) `TkSK` [14] that combines global keyword set similarity and spatial similarly, ii) `AkQST` [19]

that combines individual keyword similarity and spatial similarity, iii) STS [13] that combines GPS points with keyword annotations, and iv) At2vec+Atten [44] that use neural networks to capture spatial-semantic similarities.

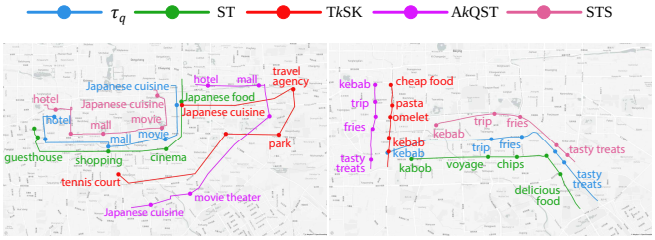
B) Search comparison. We refer to our method as the DSTSS. To evaluate the usefulness of the local optimization on ST-HNSW, we remove that from DSTSS, resulting in a variant called DSTSS w/o LO. Then, we compare these two methods with i) ID-Force that uses HashPartitioner and local HNSW-ST index to achieve distributed searches; ii) Dis-PSTSJ that is a distributed implementation of PSTSJ [13] that uses invert indexes for pruning; (iii) DITA-S that is an extension to semantic trajectory search based on DITA [31]; iv) a distributed implementation of At2vec+Atten [44], and v) LSTS-Join [21] that is the one and only existing work that was designed for distributed semantic trajectory similarity joins, which builds a hierarchical index. All the baselines are implemented with Spark.

Evaluation metrics. We use six metrics. i) **HR@k**. It denotes the proportion of results that are true positives. The ground truth is the search results via brute-force search. ii) **index time**. It refers to the time cost of index construction. iii) **Running time**. It refers to to the time cost of query process. iv) **Memory usage.** It denotes the total memory resource occupation of the distributed cluster during trajectory similarity searches. v) C_m**.** It denotes the ratio of distance calculations after pruning to the calculations in brute-force search. vi) ***SD*.** It denotes the standard deviation of the number of trajectories in each partition.

For potential studies, all codes, datasets, as well as an extended version of this manuscript including more extensions, are released at https://github.com/ZJU-DAILY/DSTSS.

B. Semantic Modeling Study (RQ1)

To verify the expressiveness of different semantic modeling methods, we select dense areas from two real-life datasets, respectively, and visualize the most similar (i.e., Top-1) trajectory to a query trajectory. The results are shown in Fig. 9, where the blue one denotes the query trajectory τ_q. Note that ST returns the same trajectory when supported by LCSS, DTW, and EDR. At2Vec+Atten returns the same trajectory as ST. Overall, we have the following observations: (i) TkSK returns a semantic-similar while spatial faraway trajectory, as TkSK mainly considers the global semantic similarities. (ii) Although AkQST considers both spatial and semantic aspects, it relies on keyword representation, thus it returns a trajectory that significantly deviates from the query trajectory. (iii) STS cannot identify semantically synonymous keywords in trajectories, as STS is based on raw representations. (iv) The search performance of At2vec+Atten is unstable. (v) Our proposed ST effectively finds the most spatial-semantic similar trajectory in the above three datasets. Note that utilizing LCSS, EDR, and DTW in ST typically yields identical returns, since these methods compute trajectory similarity via point-to-point matching. It is because of their stability that they have been widely used in trajectory similarity studies, which proves the rationality of the framework's support for these metrics.



(a) T-Drive (b) GeoLife

Fig. 9. Expressiveness of Different Semantic Aware Distance Measures

C. Search Processing Study (RQ2 & RQ3)

Effectiveness. First, we evaluate the search effectiveness, as the adopted local ST-HNSW search is based on approximation processing. We compare the proposed two methods, i.e., DSTSS and DSTSS w/o LO, with ID-Force, Dis-PSTSJ, DITA-S, At2vec+Atten, and LSTS-Join, in terms of HR@k at different k values. The results are shown in Figs. 10(a)–(c). Compared to exact search methods (e.g., Dis-PSTSJ, DITA-S, and LSTS-Join), our methods demonstrate a slight decrease in hit ratio, which can be attributed to the ST-HNSW-based approximate search. However, this reduction remains within an acceptable range. In contrast, when compared to learning-based approximate search methods (e.g., At2vec+Atten), our methods achieve a higher hit ratio, reflecting superior similarity search quality. After removing ST-HNSW and using brute-force search locally, the running time of DSTSS w/o ST-HNSW increased by 53.1×–120.8×. This is because ST-HNSW significantly improves the efficiency of local search. Compared to DSTSS w/o ST-HNSW, the reduction rate in HR@k for DSTSS is between 28.6%–35.1%. Compared with approximate search algorithms like At2Vec+Atten, this already represents a 19.7%–54.3% reduction in the reduction rate, indicating that ST-HNSW can achieve substantial efficiency gains within an acceptable effectiveness reduction rate. Finally, our methods show stable performance at different k values.

Efficiency. Then, we study the search efficiency, corresponding to the above effectiveness evaluation. The results are shown in Figs. 10(d)–(f). As observed, our methods outperform baselines up to 1–3 orders of magnitude, due to the effective computation-aware partitioning. With the increase of k, the query time of baselines increases while the query time of our methods keeps stable, which shows the superiority of our methods. This is primarily due to our proposed two-stage partitioning method, which effectively filters out a substantial number of irrelevant partitions, thereby significantly reducing the data volume for similarity searches. Furthermore, the ST-HNSW approach further minimizes the number of distance computations required between trajectories. And DSTSS continuously performs better than DSTSS w/o LO, proving the usefulness of the local optimization techniques on ST-HNSW. DSTSS has a slightly lower hit rate compared to DSTSS w/o LO, but it shows a significant reduction in running time. Because the grid optimization in the local index may miss a small number of adjacent trajectories, but it greatly reduces the number of distance calculations between trajectories. Our methods trade a slight effectiveness drop for much better efficiency, i.e., 1–3 orders of magnitude efficiency improvements.

Scalability. Next, we study the scalability performance

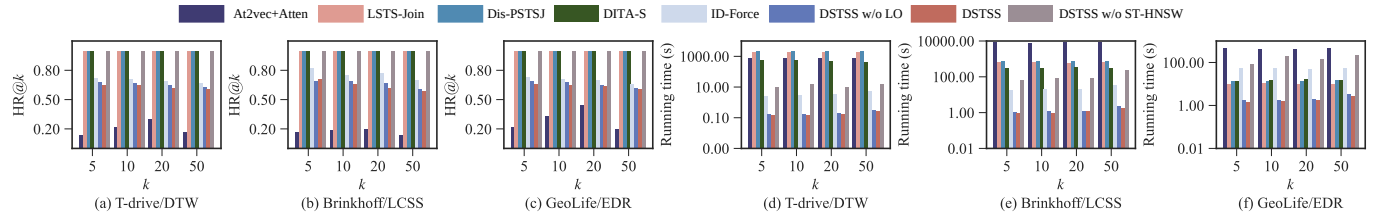


Fig. 10. Effectiveness and Efficiency Evaluation vs. k

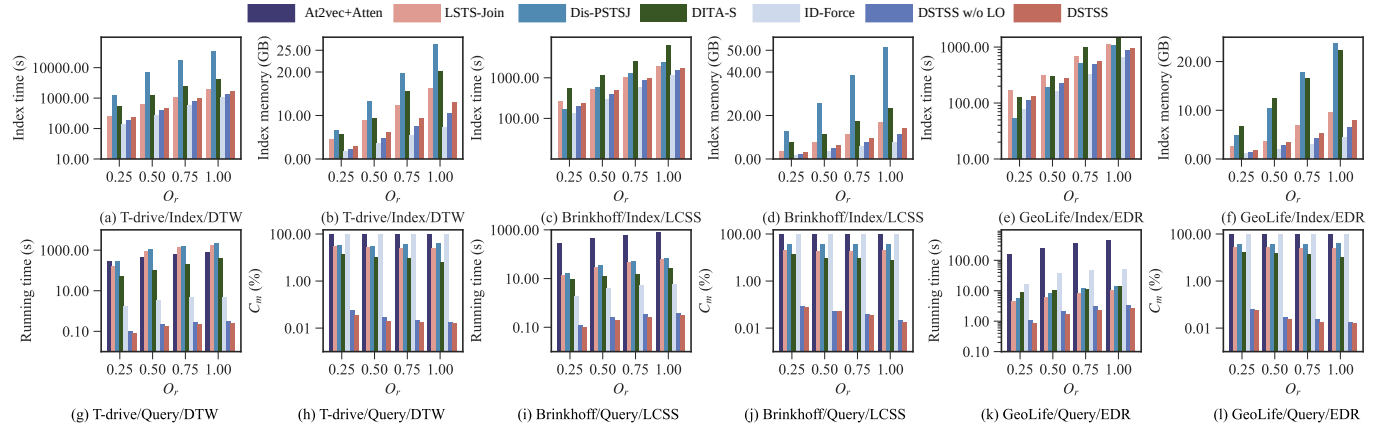


Fig. 11. Scalability Evaluation vs. Data sizes O_r

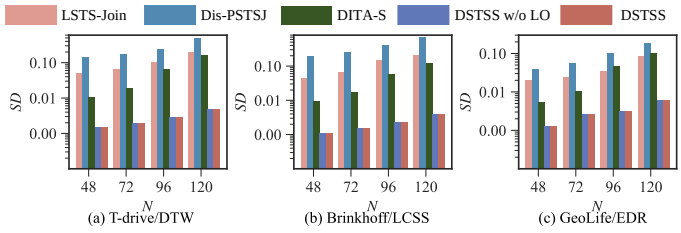


Fig. 12. Workload Evaluation

under different data sizes O_r. The results are shown in Fig. 11, where we study both index construction and query phases. In the index construction phase, i.e., Figs. 11(a)–(f), `Dis-PSTSJ` and `LSTS-Join` require more construction time and memory usage because they need to build a large inverted index of keywords, resulting in a lot of overlapping between inverted files. Second, when O_r increases, both index construction time and memory usage increase, as there are more trajectories. Note that `At2Vec+Atten` requires massive model training time (e.g., 8.55 hours on T-drive), which is omitted here. In the query phase, i.e., Figs. 11(g)–(l), `At2vec+Atten` requires the max query time because it requires tensor calculation. Second, `Dis-PSTSJ`, `DITA-S`, and `LSTS-Join` also require a long query time because they need to traverse and calculate similarity scores for a large set of trajectory candidates. Finally, `DSTSS` and `DSTSS w/o LO` show significantly less query time by reducing a large number of unnecessary similarity calculations. `DSTSS` has a slightly higher index time and index memory usage compared to `DSTSS w/o LO`, but it shows a significant reduction in running time. Finally, Figs. 11(h), 10(j), 10(l) show the corresponding pruning process, where our methods show a more powerful pruning ability than baselines beacause of the two-phase and computation-aware partitioning strategy.

Workload. Finally, we evaluate the workload by computing the load standard deviation (*SD*). Fig. 12 depicts SD by varying partition number N. As expected, our partition strategy achieves much better balancing, i.e., 1–2 orders of magnitude

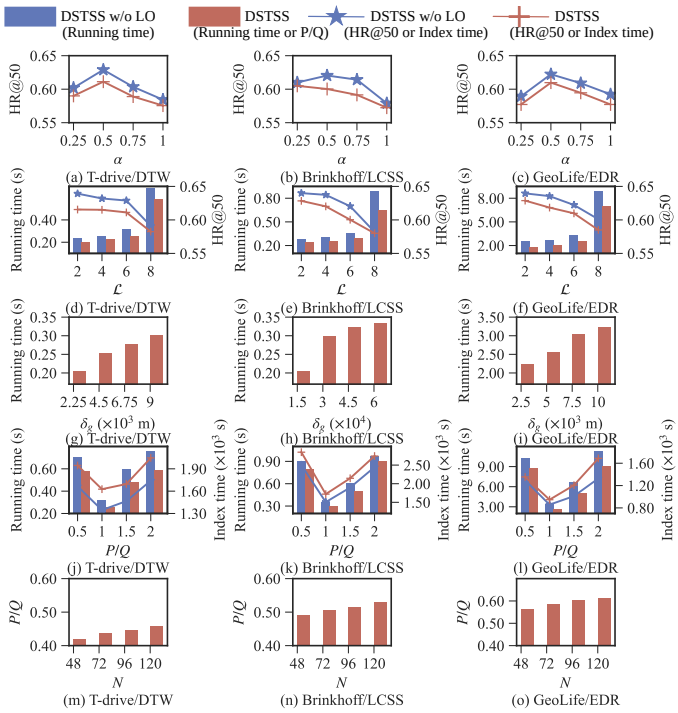


Fig. 13. Parameter Sensitivity Study

gains. This is because the DPiSAX-based semantic partitioning and the MBR-based spatial partitioning ensure the balance of data across different partitions.

D. Parameter Sensitivity Study (RQ4)

Effect of α. Figs. 13(a)–(c) report the HR@50 of `DSTSS` and `DSTSS w/o LO` by varying α. As α increases, the HR@50 on three datasets first increases and then decreases. When α is small, the effect of the semantic similarity is small; when α is large, the effect of the spatial similarity is small. Thus, when α takes an appropriate weight between 0 and 1, HR@50 reaches its peak.

Effect of $\mathcal{L}$. Figs. 13(d)–(f) show the effect of varying $\mathcal{L}$. As $\mathcal{L}$ increases, the length of the segmented geographic

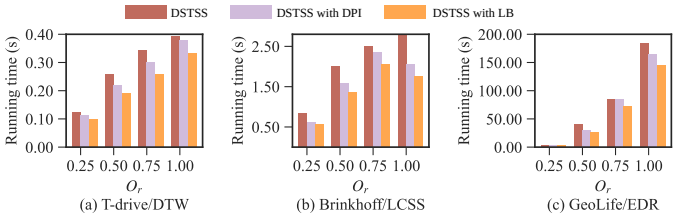


Fig. 14. Dynamic Partitioning and Indexing Evaluation vs. O_r

sequence becomes smaller, the lower bound of the encoded geographical sequence distance becomes looser, the pruning effect of spatial partitioning becomes worse, and the load becomes more unbalanced, resulting in the decline of HR@50 and the increase of query time. Therefore, it is advisable to choose a smaller segment length $\mathcal{L}$.

Effect of δ_g**.** Figs. 13(g)–(i) plot the results by varying δ_g. As δ_g increases, the query time also increases. When δ_g is very large, the effect of grid optimization on HNSW becomes weak, because it is difficult for the grid to locate entry points close to the query. Therefore, we recommend selecting the smallest grid size, while ensuring that each grid contains points.

Effect of P/Q**.** Figs. 13(j)–(l) plot the results by varying P/Q. When one of P or Q is very small while the other is large, both running and index time will increase. This is because a partitioner with a smaller number of partitions has reduced pruning capabilities, and the load becomes more imbalanced. When $P = Q$, we yield the best performance. We have also conducted parameter experiments on cluster scalability N and partition numbers P/Q to analyze the relationship between partition numbers (P/Q) and cluster scalability. Under different configurations of N, we test the optimal value of P/Q that minimizes the system's running time. As shown in Figs. 13(m)–(o), the optimal value of P/Q generally remains around 1, as an excessively high P value reduces the pruning ability of spatial partitioning, while an excessively high Q value weakens the pruning capability of semantic partitioning. When N increases, the optimal P/Q decreases slightly, indicating that spatial partitioning exhibits stronger pruning effectiveness under larger cluster scales.

E. Dynamic Scenario Study (RQ5)

We have also evaluated the extension of dynamic partitioning and indexing by varying the parameters O_r, with the experimental results shown in Figs. 14(a)–(c). The experiments were conducted on SparkStreaming. As can be seen, the running time of DSTSS with dynamic partitioning and indexing (`DSTSS with DPI`) for queries is 4.3%–25.6% shorter than that of `DSTSS` and `DSTSS w/o LO`. This is because, in scenarios involving dynamic data insertion, the absence of dynamic partitioning leads to gradually increasing skew in the current partitions. During distributed computation, the running time is determined by the partition with the largest data volume, which significantly increases the overall running time. Moreover, as the total data volume O_r grows, the advantage becomes more pronounced, since the impact of data skew becomes more severe with larger data volumes.

We have also evaluated the method of load balancing oriented partitioning by varying the parameters O_r, with the experimental results shown in Figs. 14(a)–(c). The experiments

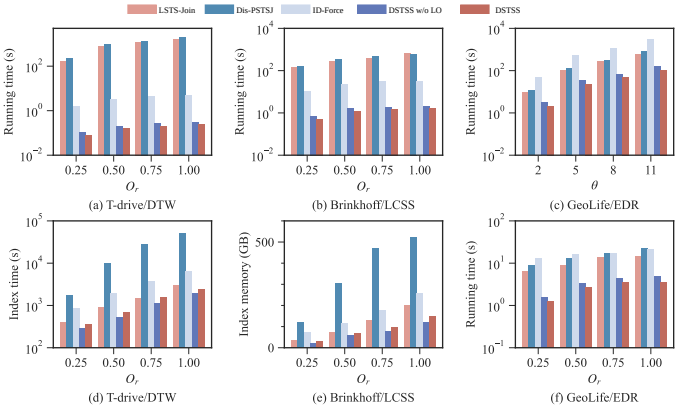


Fig. 15. Dynamic Partitioning and Indexing Evaluation vs. O_r

were also conducted on SparkStreaming to evaluate in the dynamic scenario. As can be seen, the running time of DSTSS with load balancing oriented partitioning (`DSTSS with LB`) is 10.7%–14.9% shorter than that of `DSTSS with DPI`. Under the mechanism of load balancing oriented partitioning, the load across various partitions in a dynamic environment can achieve balance more quickly, thereby reducing the running time. Additionally, as the total data volume O_r increases, the advantages of load balancing oriented partitioning become more pronounced. This is because, with larger data volumes, the impact of data skew becomes more significant, making load balancing even more critical to ensure high throughput and low latency.

F. Extensions Study (RQ6)

Finally, we also prove that our framework supports other trajectory similarity queries (i.e., threshold-based similarity query), which can be found in the complete manuscript [55].

Then, we show that how `DSTSS` performs when it extends to the threshold-based search and time-ordered search.

Threshold-based Search. When it extends to the threshold-based search, we perform a set of experiments by varying the parameters θ and O_r. The results are shown in Figs. 15(a)–(c). As observed, `DSTSS` outperform baselines up to 1–3 orders of magnitude, due to the similar computation and pruning process with kNN search. As θ increases, the running time grows rapidly because it requires evaluating and returning a large number of trajectories. This leads to a deterioration in the pruning effect of all methods, while our method, although experiencing a performance decline, still maintains great superiority over the baselines.

Time-ordered Search. When it extends to the time-ordered search, we also perform a set of experiments by varying the parameters O_r. The results are shown in Figs. 15(d)–(f). `DSTSS` maintains strong performance. The inclusion of the time dimension does not impact the partitioning strategy or the local ST-HNSW index. It merely necessitates additional distance calculations for the time dimension during coordinate point distance computation.

IX. Related Work

(A) Semantic Trajectory Representation. Existing semantic trajectory representation methods can be categorized into individually keywords-tagged representation [13], [14],

[19], [22], [39], [56]–[60] and globally keywords-tagged representation [40], [41], [61], [62]. In an individual-based trajectory, a set of keywords is assigned to each point. In contrast, a global-based trajectory assigns keywords to the entire trajectory. Nevertheless, these methods possess limitations. First, they can only capture semantics based on keywords, disregarding other forms of semantics such as text. Second, the global-based representation fails to preserve the order of semantics, while the individual-based representation leads to redundant storage [20]. Last but not the least, keyword-based representation typically necessitates the construction of an inverted index using keywords [13]. In that case, as the number of keywords increases, the redundancy escalates. There are also works to attach keywords to road network constrained trajectories [20], [63], which is not the scope of this paper. Recently, [64], [65] introduced an approach for semantic trajectory similarity calculation that totally does not consider the spatial aspect in measuring inter-trajectory similarity. In this paper, we propose a new representation by decoupling the semantic trajectory into geographical and semantic sequences, respectively. This not only resolves the issue of redundant storage but also preserves the sequential characteristics.

(B) Semantic Trajectory Similarity Searches. The objective of the similarity search is to retrieve trajectories that are similar to a specified query trajectory. There are two types of solutions. The first type [13], [14], [19], [21], [64]–[66] relies on keyword-based inverted indexes. They utilize the inverted indexes to create global indexes for semantic trajectories and then construct local spatial indexes such as R-trees. Ultimately, the similarity search is performed on local indexes. However, they suffer from storage redundancy due to the trajectories appearing repeatedly in different inverted files. The second type [11], [43], [44], [63] is based on deep learning. Specifically, they employ deep models like Seq2seq and Attention to embed the whole semantic trajectories into vectors. Then, they perform searches in the vector space. However, the learning-based methods fail to provide interpretable results. To the best of our knowledge, there is no work that utilizes a distributed approach for semantic-spatial trajectory search except [21]. However, it still relies on keyword-based inverted index and the pruning method lacks efficiency.

X. Conclusion

In this paper, we propose a distributed framework for semantic trajectory similarity search. Specifically, we propose a new semantic representation method to enable robust and multi-attribute semantic modeling. Then, we develop a computation-aware two-phase partitioning architecture so that on a query trajectory, most of the data partitions can be efficiently pruned. In each partition, we develop several optimization techniques based on ST-HNSW index to further speed up. Experiments using real and synthetic datasets show the superiority of our methods. In the future, it is interesting to include more types of semantics, like geo-photos and videos.

References

[1] R. Li, H. He, R. Wang, S. Ruan, T. He, J. Bao, J. Zhang, L. Hong, and Y. Zheng, "Trajmesa: A distributed nosql-based trajectory data management system," *TKDE*, vol. 35, no. 1, pp. 1013–1027, 2023.

[2] J. Dai, B. Yang, C. Guo, and Z. Ding, "Personalized route recommendation using big trajectory data," in *ICDE*, 2015, pp. 543–554.

[3] B. Zheng, L. Weng, X. Zhao, K. Zeng, X. Zhou, and C. S. Jensen, "REPOSE: distributed top-k trajectory similarity search with local reference point tries," in *ICDE*, 2021, pp. 708–719.

[4] S. Wang, Z. Bao, J. S. Culpepper, T. Sellis, and X. Qin, "Fast large-scale trajectory clustering," *VLDB*, vol. 13, no. 1, pp. 29–42, 2019.

[5] L. Chen, Y. Gao, Z. Fang, X. Miao, C. S. Jensen, and C. Guo, "Real-time distributed co-movement pattern detection on streaming trajectories," *VLDB*, vol. 12, no. 10, pp. 1208–1220, 2019.

[6] Z. Luo, L. Li, M. Zhang, W. Hua, Y. Xu, and X. Zhou, "Diversified top-k route planning in road network," *VLDB*, vol. 15, no. 11, pp. 3199–3212, 2022.

[7] H. Zhang, X. Zhang, Q. Jiang, B. Zheng, Z. Sun, W. Sun, and C. Wang, "Trajectory similarity learning with auxiliary supervision and optimal matching," in *IJCAI*, 2020, pp. 3209–3215.

[8] P. Yang, H. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "T3S: effective representation learning for trajectory similarity computation," in *ICDE*, 2021, pp. 2183–2188.

[9] D. Yao, H. Hu, L. Du, G. Cong, S. Han, and J. Bi, "Trajgat: A graph-based long-term dependency modeling approach for trajectory similarity computation," in *KDD*, 2022, pp. 2275–2285.

[10] J. Ding, J. Fang, Z. Zhang, P. Zhao, J. Xu, and L. Zhao, "Real-time trajectory similarity processing using longest common subsequence," in *HPCC*, 2019, pp. 1398–1405.

[11] Y. Zhang, A. Liu, G. Liu, Z. Li, and Q. Li, "Deep representation learning of activity trajectory similarity computation," in *ICWS*, E. Bertino, C. K. Chang, P. Chen, E. Damiani, M. Goul, and K. Oyama, Eds., 2019, pp. 312–319.

[12] H. Zhu, W. Li, W. Liu, J. Yin, and J. Xu, "Top k optimal sequenced route query with POI preferences," *Data Sci. Eng.*, vol. 7, no. 1, pp. 3–15, 2022.

[13] L. Chen, S. Shang, C. S. Jensen, B. Yao, and P. Kalnis, "Parallel semantic trajectory similarity join," in *ICDE*, 2020, pp. 997–1008.

[14] K. Zheng, B. Zheng, J. Xu, G. Liu, A. Liu, and Z. Li, "Popularity-aware spatial keyword search on activity trajectories," *WWW*, vol. 20, no. 4, pp. 749–773, 2017.

[15] Google, "Google map support," [Online], 2025, https://support.google.com/mymaps.

[16] Foursquare, "Foursquare," [Online], 2025, https://foursquare.com.

[17] Wanderlog, "One app for all your travel planning needs," [Online], 2025, https://wanderlog.com.

[18] Rome2rio, "Discover how to get anywhere," [Online], 2025, https://www.rome2rio.com.

[19] B. Zheng, N. J. Yuan, K. Zheng, X. Xie, S. W. Sadiq, and X. Zhou, "Approximate keyword search in semantic trajectory database," in *ICDE*, 2015, pp. 975–986.

[20] H. Issa and M. L. Damiani, "Efficient access to temporally overlaying spatial and textual trajectories," in *MDM*, C. Chow, P. P. Jayaraman, and W. Wu, Eds., 2016, pp. 262–271.

[21] R. Tian, J. Li, W. Zhang, and F. Wang, "A distributed framework for large-scale semantic trajectory similarity join," *Multim. Tools Appl.*, vol. 83, no. 6, pp. 16 205–16 229, 2024.

[22] K. Zheng, S. Shang, N. J. Yuan, and Y. Yang, "Towards efficient search for activity trajectories," in *ICDE*, 2013, pp. 230–241.

[23] B. Yi, H. V. Jagadish, and C. Faloutsos, "Efficient retrieval of similar time sequences under time warping," in *ICDE*, 1998, pp. 201–208.

[24] M. Vlachos, D. Gunopulos, and G. Kollios, "Discovering similar multidimensional trajectories," in *ICDE*, 2002, pp. 673–684.

[25] L. Chen, M. T. Özsu, and V. Oria, "Robust and fast similarity search for moving object trajectories," in *SIGMOD*, 2005, pp. 491–502.

[26] J. Yuan, Y. Zheng, X. Xie, and G. Sun, "Driving with knowledge from the physical world," in *KDD*, 2011, pp. 316–324.

[27] Y. Zheng, Q. Li, Y. Chen, X. Xie, and W. Ma, "Understanding mobility based on GPS data," in *UbiComp*, 2008, pp. 312–321.

[28] AMap, "Amap open platform," [Online], 2025, https://lbs.amap.com/.

[29] D. Xie, F. Li, and J. M. Phillips, "Distributed trajectory similarity search," *VLDB*, vol. 10, no. 11, pp. 1478–1489, 2017.

[30] Z. Zhang, X. Qi, Y. Wang, C. Jin, J. Mao, and A. Zhou, "Distributed top-k similarity query on big trajectory streams," *Frontiers Comput. Sci.*, vol. 13, no. 3, pp. 647–664, 2019.

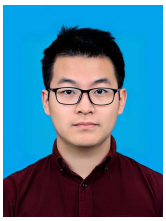
[31] Z. Shang, G. Li, and Z. Bao, "DITA: distributed in-memory trajectory analytics," in *SIGMOD*, 2018, pp. 725–740.

[32] M. A. Zaharia, R. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. E. Gonzalez, S. Shenker, and I. Stoica, "Apache spark," *Communications of the ACM*, vol. 59, pp. 56 – 65, 2016.

[33] H. Yuan and G. Li, "Distributed in-memory trajectory similarity search and join on road network," in *ICDE*, 2019, pp. 1262–1273.
[34] M. A. Nascimento and J. R. O. Silva, "Towards historical r-trees," in *SAC*, 1998, pp. 235–240.
[35] Y. Tao and D. Papadias, "Mv3r-tree: A spatio-temporal access method for timestamp and interval queries," in *VLDB 2001, Proceedings of 27th International Conference on Very Large Data Bases, September 11-14, 2001, Roma, Italy*, 2001, pp. 431–440.
[36] P. Zhou, D. Zhang, B. Salzberg, G. Cooperman, and G. Kollios, "Close pair queries in moving object databases," in *ACM-GIS*, 2005, pp. 2–11.
[37] V. P. Chakka, A. Everspaugh, and J. M. Patel, "Indexing large trajectory data sets with SETI," in *CIDR*, 2003.
[38] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
[39] G. Cong, H. Lu, B. C. Ooi, D. Zhang, and M. Zhang, "Efficient spatial keyword search in trajectory databases," *CoRR*, vol. abs/1205.2880, 2012.
[40] S. Shang, R. Ding, B. Yuan, K. Xie, K. Zheng, and P. Kalnis, "User oriented trajectory search for trip recommendation," in *EDBT*, 2012, pp. 156–167.
[41] C. Zhu, J. Xu, C. Liu, P. Zhao, A. Liu, and L. Zhao, "Efficient trip planning for maximizing user satisfaction," in *DASFAA*, ser. Lecture Notes in Computer Science, M. Renz, C. Shahabi, X. Zhou, and M. A. Cheema, Eds., vol. 9049, 2015, pp. 260–276.
[42] T. Boonchoo, X. Ao, and Q. He, "Multi-aspect embedding for attribute-aware trajectories," *Symmetry*, vol. 11, no. 9, p. 1149, 2019.
[43] C. Gao, Z. Zhang, C. Huang, H. Yin, Q. Yang, and J. Shao, "Semantic trajectory representation and retrieval via hierarchical embedding," *Inf. Sci.*, vol. 538, pp. 176–192, 2020.
[44] A. Liu, Y. Zhang, X. Zhang, G. Liu, Y. Zhang, Z. Li, L. Zhao, Q. Li, and X. Zhou, "Representation learning with multi-level attention for activity trajectory similarity computation," *TKDE*, vol. 34, no. 5, pp. 2387–2400, 2022.
[45] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *EMNLP-IJCNLP*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., 2019, pp. 3980–3990.
[46] D. E. Yagoubi, R. Akbarinia, F. Masseglia, and T. Palpanas, "Massively distributed time series indexing and querying," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 1, pp. 108–120, 2020.
[47] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *ACM Symposium on Computational Geometry*, 2004, pp. 253–262.
[48] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization for approximate nearest neighbor search," in *CVPR*, 2013, pp. 2946–2953.
[49] J. Li, H. Liu, C. Gui, J. Chen, Z. Ni, and N. Wang, "The design and implementation of a real time visual search system on JD e-commerce platform," *CoRR*, vol. abs/1908.07389, 2019.
[50] P. Ciaccia, M. Patella, and P. Zezula, "M-tree: An efficient access method for similarity search in metric spaces," in *VLDB*, 1997, pp. 426–435.
[51] E. J. Keogh, "Exact indexing of dynamic time warping," in *VLDB*, 2002, pp. 406–417.
[52] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Inf. Syst.*, vol. 45, pp. 61–68, 2014.
[53] T. Brinkhoff, "A framework for generating network-based moving objects," *GeoInformatica*, vol. 6, no. 2, pp. 153–180, 2002.
[54] T. Sahni, C. Chandak, N. R. Chedeti, and M. Singh, "Efficient twitter sentiment classification using subjective distant supervision," in *COMSNETS*, 2017, pp. 548–553.
[55] S. Gong, Z. Fang, L. Chen, L. Liu, Y. Gao, K. Zheng, and G. Chen, https://github.com/ZJU-DAILY/DSTSS, 2023.
[56] C. Cayèré, C. Sallaberry, C. Faucher, M. Bessagnet, P. Roose, M. Masson, and J. Richard, "Multi-level and multiple aspect semantic trajectory model: Application to the tourism domain," *ISPRS*, vol. 10, no. 9, p. 592, 2021.
[57] Y. Cao, F. Xue, Y. Chi, Z. Ding, L. Guo, Z. Cai, and H. Tang, "Effective spatio-temporal semantic trajectory generation for similar pattern group identification," *Int. J. Mach. Learn. Cybern.*, vol. 11, no. 2, pp. 287–300, 2020.
[58] H. Noureddine, C. Ray, and C. Claramunt, "Semantic trajectory modelling in indoor and outdoor spaces," in *MDM*, 2020, pp. 131–136.
[59] A. P. R. de Souza, C. Renso, R. Perego, and V. Bogorny, "Mat-index: An index for fast multiple aspect trajectory similarity measuring," *Trans. GIS*, vol. 26, no. 2, pp. 691–716, 2022.
[60] J. Zhu, D. Cheng, W. Zhang, C. Song, J. Chen, and T. Pei, "A new approach to measuring the similarity of indoor semantic trajectories," *ISPRS*, vol. 10, no. 2, p. 90, 2021.
[61] K. Liu, B. Yang, S. Shang, Y. Li, and Z. Ding, "MOIR/UOTS: trip recommendation with user oriented trajectory search," in *MDM*, vol. 1, 2013, pp. 335–337.
[62] P. Wang, J. Yang, and J. Zhang, "Location prediction for indoor spaces based on trajectory similarity," in *DSIT*, 2021, pp. 402–407.
[63] J. Jiang, D. Pan, H. Ren, X. Jiang, C. Li, and J. Wang, "Self-supervised trajectory representation learning with temporal regularities and travel semantics," in *ICDE*, 2023, pp. 843–855.
[64] D. R. de Almeida, C. de Souza Baptista, and F. G. de Andrade, "Similarity search on semantic trajectories using text processing," *ISPRS*, vol. 11, no. 7, p. 412, 2022.
[65] C. Cai and D. Lin, "Find another me across the world - large-scale semantic trajectory analysis using spark," *TKDE*, vol. 35, no. 9, pp. 8905–8918, 2023.
[66] J. B. Rocha-Junior, O. Gkorgkas, S. Jonassen, and K. Nørvåg, "Efficient processing of top-k spatial keyword queries," in *SSTD*, ser. Lecture Notes in Computer Science, vol. 6849, 2011, pp. 205–222.



Shenghao Gong received the BS degree in computer science from Xiamen University, China, in 2021. He is currently working toward the PhD degree in School of Software Technology, Zhejiang University, China. His research interests include big data analytics and spatial-temporal database.



Ziquan Fang received the PhD degree in computer science from Zhejiang University, China, in 2023. He is currently a ZJU Plan 100 Professor in the School of Software, Zhejiang University, China. His research interests include distributed big trajectory data management and analytics, spatio-temporal data mining, and intelligent traffic systems.



Liu Liu received his BS degree in Software Engineering from Zhejiang University, China, in 2023. He is pursuing an MS degree at the College of Computer Science and Technology, Zhejiang University. His research interests include stream processing, distributed computing, and auto-tuning systems.



Yaofeng Tu received the PhD degree from Nanjing University of Aeronautics and Astronautics, China, in 2019. He is currently a Research Professor with ZTE Corporation. His main research interests include distributed computing, big data systems, and machine learning.



Yunjun Gao received the PhD degree in computer science from Zhejiang University, China, in 2008. He is currently a professor in the College of Computer Science, Zhejiang University, China. His research interests include spatial and spatio-temporal databases, metric and incomplete/uncertain data management, graph databases, spatio-textual data processing, and database usability. He is a member of the ACM and the IEEE.